

Concepteur Développeur d'Applications

Niveau 6 (Bac +3)

Dossier de projet professionnel présenté dans le cadre de la certification RNCP

LECART Christophe

Zone01 Rouen Normandie

Session d'examen JUIN 2026

Référentiel RNCP : 37873

Konekt

DÉVELOPPEMENT D'UNE PLATEFORME
SOCIALE FULL-STACK AVEC MESSAGERIE
EN TEMPS RÉEL, GROUPES, ÉVÈNEMENTS
ET NOTIFICATIONS

Durée du projet : 26 mai 2025 – 18 septembre 2025

SOMMAIRE

1. PRESENTATION DU CANDIDAT.....	3
1.1 PARCOURS ET RECONVERSION.....	3
1.2 COMPETENCES MOBILISEES.....	4
1.3 VISION PROFESSIONNELLE.....	4
2. CONTEXTE DU PROJET.....	5
2.1 GENESE ET MOTIVATION.....	5
2.2 PERIMETRE FONCTIONNEL.....	5
2.3 ENJEUX TECHNIQUES ET PEDAGOGIQUES.....	6
2.4 ORGANISATION DU PROJET – GITHUB PROJECTS (KANBAN).....	7
3. OBJECTIFS DU PROJET.....	9
3.1 OBJECTIFS TECHNIQUES.....	9
3.2 OBJECTIFS CDA / RNCP.....	10
4. SPECIFICATIONS FONCTIONNELLES.....	11
4.1 ANALYSE DU BESOIN UTILISATEUR.....	11
4.2 USER STORIES – SELECTION REPRESENTATIVE.....	13
4.3 FONCTIONNALITES PRINCIPALES.....	15
5. SPECIFICATIONS TECHNIQUES.....	17
5.0 CONCEPTION UML, MERISE ET SQL.....	17
5.1 ARCHITECTURE LOGICIELLE GLOBALE.....	20
5.2 ARCHITECTURE LOGICIELLE SIMPLIFIEE.....	20
5.3 ORGANISATION DES FICHIERS – APP ROUTER.....	21
5.4 TECHNOLOGIES UTILISEES.....	23
5.5 MODELE DE DONNEES – 18 ENTITES PRISMA.....	25
6. REALISATIONS TECHNIQUES.....	28
6.0 ECRANS IMPLEMENTES.....	28
6.1 AUTHENTIFICATION JWT + OAUTH GOOGLE.....	29
6.2 API REST – 60+ ENDPOINTS.....	30

6.3 MESSAGERIE TEMPS REEL – SSE+REDIS.....	31
6.4 SYSTEMES DE REACTIONS UNIVERSELLES.....	33
6.5 GROUPES, EVENEMENTS ET NOTIFICATIONS.....	35
6.6 PERFORMANCES – OPTIMISATION DES REQUETES PRISMA.....	37
7. TESTS ET VALIDATION.....	38
7.1 STRATEGIE DE TEST.....	38
7.2 JEUX D’ESSAI - AUTHENTIFICATION.....	38
7.3 JEUX D’ESSAI – MESSAGERIE ET TEMPS REEL.....	39
7.4 JEUX D’ESSAI – SECURITE.....	39
7.5 JEUX D’ESSAI – POSTS ET INTERACTIONS.....	40
7.6 GROUPES ET EVENEMENTS.....	40
7.7 RESULTAT GLOBAL – SYNTHESE DES TESTS.....	41
8. SECURITE ET RGPD.....	42
8.1 MESURES DE SECURITE IMPLEMENTEES.....	42
8.2 CONFORMITE RGPD.....	42
9. DEPLOIEMENT.....	45
9.1 INFRASTRUCTURE DOCKER – DEVELOPPEMENT LOCAL.....	45
9.2 CI/CD ET MISE EN PRODUCTION.....	46
10. RECHERCHE TECHNIQUE PERSONNELLE.....	48
10.1 APPROCHE DE VEILLE.....	48
10.2 ANALYSE COMPARATIVE – TECHNOLOGIES TEMPS REEL.....	49
10.3 SOURCES MOBILISEES.....	49
11. BILAN RNCP ET CONCLUSION.....	50
11.1 COUVERTURE DES COMPETENCES RNCP 37873.....	50
11.2 BILAN PERSONNEL.....	51
11.3 FORCES ET AXES D’AMELIORATION.....	51
11.4 VISION ET PERSPECTIVES.....	52

1. PRESENTATION DU CANDIDAT

1.1 PARCOURS ET RECONVERSION

Je m'appelle Christophe LECART, j'ai 44 ans et je suis actuellement en formation à Zone01 Rouen Normandie dans le cadre de l'obtention du diplôme de « Concepteur Développeur d'Applications » (niveau 6 - Bac+3, RNCP 37873).

Mon parcours est marqué par une reconversion professionnelle après plus de vingt ans passés dans l'industrie, notamment au sein de groupes comme SOCOTEC (inspection technique, gestion de la qualité) et KONE (systèmes d'ascenseurs, maintenance industrielle). Ces expériences m'ont permis d'acquérir une rigueur opérationnelle, un sens de la responsabilité, une capacité à travailler sur des systèmes techniques complexes et le respect de procédures strictes.

J'ai intégré Zone01 Rouen Normandie en 2024, une formation fondée sur l'autonomie, le travail en peer-to-peer et l'apprentissage par la pratique. Ce modèle pédagogique me permet de développer une approche structurée du code : organisation de la logique applicative, écriture de code maintenable et travail sur des projets de bout en bout.

Parcours professionnel

PERIODE	POSTE / STRUCTURE	COMPETENCES TRANSFEREES
2002 – 2003	LUBRIZOL - Technicien bureau d'études (alternance) -	Support technique et participation à des activités d'études
2003 – 2010	KONE - Technicien hautement qualifié -	Travail en environnement technique, respect des procédures, interventions sur systèmes industriels
2010 – 2024	SOCOTEC - Technicien / Expert -	Gestion de missions techniques, conformité, analyse réglementaire, travail en équipe avec ingénieurs
2024 – aujourd'hui	ZONEO1 ROUEN NORMANDIE - Formation -	Full-stack TypeScript, architecture logicielle, sécurité, DevOps

1.2 COMPETENCES MOBILISEES

Au cours du projet Konekt, nous avons mobilisé un ensemble de compétences couvrant l'intégralité du cycle de développement d'une application web moderne, depuis la conception de l'architecture jusqu'au déploiement en production.

DOMAINES	TECHNOLOGIES / PRATIQUES
Frontend	Next.js 15 App Router, React 19, TypeScript, Tailwind CSS
Backend / API	Next.js API Routes, REST, validation Zod, 60+ endpoints
Base de données	Prisma ORM, PostgreSQL (Neon), 18 modèles, 40+ relations
Temps réel	Server-Sent Events (SSE), Redis Upstash, polling fallback
Sécurité	JWT (jose), bcrypt, middleware auth, headers sécurisés
Cloud / Media	Cloudinary (upload images), Neon DB serverless PostgreSQL
DevOps	Docker, Docker Compose, Vercel (déploiement Git natif)
Qualité	Tests intégration, jeux d'essai, Playwright E2E

1.3 VISION PROFESSIONNELLE

Ce projet marque pour moi un tournant : je ne suis plus simplement quelqu'un qui code, mais un développeur capable de concevoir, architecturer, sécuriser et livrer une application complexe de A à Z.

Mon profil est à la fois technique et pragmatique, influencé par mon expérience professionnelle passée à faire fonctionner des systèmes réels et complexes dans des contextes exigeants.

Je me positionne comme développeur full-stack TypeScript, avec une attirance particulière pour l'architecture back-end, la performance des requêtes, la sécurité des API et les systèmes temps réel.

Je suis actuellement en recherche d'alternance de niveau Bac+5 pour consolider et approfondir cette trajectoire après l'obtention de mon diplôme.

2. CONTEXTE DU PROJET

2.1 GENESE ET MOTIVATION

Le projet Konekt est né d'une volonté de construire une application ambitieuse couvrant l'ensemble du spectre technique attendu d'un CDA : architecture API complexe, gestion d'état, sécurité robuste et fonctionnalités temps réel avec une mise en production simulée grâce à Vercel et une conteneurisation avec Docker et Dockerfile.

Plutôt que de choisir un projet fictif ou un exercice pédagogique simplifié, nous avons opté pour la construction d'une plateforme sociale fonctionnelle à part entière, comparable à des applications déjà existantes comme Facebook, Instagram ou consorts — non pas dans l'ambition commerciale, mais dans la complexité technique.

Ce choix nous a confronté à des problèmes réels : gestion des performances sur des requêtes N+1, cohérence des données dans un système multi-entités, synchronisation temps réel sans WebSocket (contrainte d'hébergement serverless), sécurisation des routes API, et orchestration de services cloud distincts.

2.2 PERIMETRE FONCTIONNEL

L'application couvre un spectre fonctionnel large, organisé en huit domaines majeurs :

- **Authentification** — Inscription, connexion JWT, OAuth Google, sessions persistantes, déconnexion, protection des routes
- **Profils utilisateurs** — Édition du profil, photo, biographie, relations de suivi (follow/unfollow), liste d'amis
- **Posts et médias** — Création de publications texte/image, fil d'actualité, réactions (7 types), commentaires imbriqués, partage
- **Stories** — Publications éphémères 24h, affichage en cercles, compteur de vues
- **Groupe**s — Création, adhésion, rôles (admin/modérateur/membre), posts de groupe, événements liés
- **Événements** — Création avec date/lieu, RSVP (aller/peut-être/ne pas y aller), liste des participants
- **Messagerie directe et de groupe** — Chat 1:1 et multi-utilisateurs, SSE temps réel, historique paginé, statut lu/non lu
- **Notifications** — Système transversal couvrant les invitations de groupe (GROUP_INVITATION) – modèle extensible via champ de type String

Métriques du projet

INDICATEUR	VALEUR	INDICATEUR	VALEUR
Modèles Prisma	18	Endpoints API	60+
Relations BDD	39	User Stories	38
Pages / Routes	18	Composants React	130+
Durée développement	~1 200h	Commits GitHub	320+
Lignes de code	~30 000	Tests automatisés	7 (TypeScript + ESLint)

2.3 ENJEUX TECHNIQUES ET PEDAGOGIQUES

Le projet présente plusieurs enjeux techniques structurants qui dépassent le simple exercice de CRUD :

- **Scalabilité des requêtes** — 18 modèles interdépendants impliquent des requêtes Prisma complexes avec includes imbriqués. La performance du fil d'actualité a nécessité plusieurs cycles d'optimisation.
- **Temps réel sans WebSocket** — Hébergé sur Vercel (serverless), l'utilisation de WebSockets n'est pas possible nativement. Nous avons implémenté un système SSE (Server-Sent Events) combiné à Redis Upstash comme bus de messages.
- **Sécurité multi-couches** — Auth JWT avec middleware Next.js, validation Zod sur tous les inputs, protection CORS, sanitisation des uploads Cloudinary, headers HTTP de sécurité.
- **Cohérence des données** — Les notifications, compteurs de likes, statuts de lecture et relations de suivi nécessitent une gestion précise des transactions Prisma pour éviter les états incohérents.
- **Conteneurisation et reproductibilité** — L'environnement Docker Compose permet de recréer l'ensemble de la stack en une commande, facilitant les tests et le déploiement.

2.4 ORGANISATION DU PROJET – GITHUB PROJECTS (KANBAN)

Le backlog du projet a été géré via GitHub Projects avec un board Kanban composé de quatre colonnes :

- **Sections** — épics en attente de découpe en sous-tâches actionnables
- **TODO** — tickets priorités en attente de développement
- **In progress** — tâches en cours de développement
- **Done** — fonctionnalités livrées et validées

Chaque ticket est qualifié selon deux labels :

- **Priorité** : P2 (haute), P3 (moyenne), P4 (faible)
- **Taille** : M (medium), L (large), XL (extra-large)

Cette double qualification permet de prioriser le travail tout en estimant la charge avant d'affecter une tâche. Les épics complexes (*Chat System, Groups & Events, Notifications*) sont découpés en sous-tâches tracées directement dans l'issue parente.

Au total : **59 issues fermées et 13 ouvertes** au moment de la soutenance, couvrant l'authentification, le feed, la messagerie temps réel, les groupes, les événements, les notifications et le déploiement.

Board Kanban — vue d'ensemble des colonnes et labels priorité/taille :

The Kanban board is organized into four columns, each with a title, a count, and an estimate of 0. The columns are:

- Sections (5):** This column contains five task cards, each representing a different system component. Each card has a green circle icon, a title, and two labels: a purple circle with 'P2' and a yellow circle with 'XL'.
- TODO (3):** This column contains three task cards. Each card has a green circle icon, a title, and a purple circle with 'P4'.
- In progress (4):** This column contains four task cards. Each card has a green circle icon, a title, and two labels: a purple circle with 'P3' and a red circle with 'M'.
- Done (56):** This column contains five task cards. Each card has a green circle icon, a title, and two labels: a purple circle with 'P2' and a red circle with 'M'.

Liste des issues — open 13 / closed 59:

The list of issues is displayed in a table-like format. The top row shows the status of the issues: 13 Open and 59 Closed. The table has columns for the issue number, title, author, and date. The issues are listed in descending order of their creation date.

Issue Number	Title	Author	Date
#111	Settings	arochet	Aug 18, 2025
#76	Internationalization	pcaboor	Jun 16, 2025
#66	Feature: Implement OAuth (Google) authentication with fallback to existing account	Rookate	Jun 4, 2025
#51	PATCH /notifications/id/read — Mark as read	Zanakan12	May 26, 2025
#46	Seed script — Demo data	Zanakan12	May 26, 2025
#45	CI — Lint, test, build, push Image	Zanakan12	May 26, 2025
#43	docker-compose.yml — App + DB	Zanakan12	May 26, 2025
#40	DevOps: Docker, CI/CD	Zanakan12	May 26, 2025
#39	Notifications	Zanakan12	May 26, 2025
#37	Chat System	Zanakan12	May 26, 2025
#30	Group feed — Posts & comments inside group	arochet	May 22, 2025
#24	Groups & Events	arochet	May 22, 2025
#13	Follow System	CLcart	May 21, 2025

3. OBJECTIFS DU PROJET

3.1 OBJECTIFS TECHNIQUES

Le projet Konekt a été défini avec un ensemble d'objectifs techniques précis, couvrant l'ensemble des couches d'une application web moderne :

1. Architecture App Router Next.js

Maîtriser le nouveau paradigme App Router de Next.js 15 avec les React Server Components, les layouts imbriqués, le streaming SSR et la séparation client/serveur.

2. Couche données relationnelle complexe

Concevoir et implémenter un schéma Prisma couvrant 18 entités avec des relations multiples (many-to-many, self-referencing, polymorphiques), et optimiser les requêtes pour éviter les problèmes N+1.

3. API REST sécurisée

Développer plus de 60 endpoints API avec authentification JWT, validation Zod, gestion des erreurs standardisée, et pagination offset (skip/take).

4. Temps réel avec contraintes serverless

Implémenter un système de messagerie et de notifications en temps réel via SSE + Redis Upstash, sans WebSocket, compatible avec l'hébergement Vercel.

5. Sécurité et authentification

Mettre en place une authentification JWT robuste avec OAuth Google, middleware de protection des routes, et gestion des sessions côté serveur.

6. Déploiement et CI/CD

Conteneuriser l'application avec Docker (multi-stage build) pour les environnements de développement (profil *dev*) et de production (profil *prod*). Le déploiement sur Vercel est déclenché automatiquement par l'intégration Git native de Vercel sur push vers *main*.

3.2 OBJECTIFS CDA / RNCP

En parallèle des objectifs techniques, le projet permet de valider l'ensemble des blocs de compétences du titre CDA (RNCP 37873) :

BLOC RNCP	COMPETENCES VISEES	REALISATIONS DANS LE PROJET
BLOC 1 Développement d'application	Développer des composants d'accès aux données, concevoir une IHM, développer la partie back-end d'une interface utilisateur web.	80+ composants React, 60+ endpoints API, Prisma ORM, validation Zod, formulaires TypeScript
BLOC 2 Architecture logicielle	Concevoir une application en couches, utiliser des patterns architecturaux, modéliser les données (MCD/MLD/MPD).	App Router Next.js, 18 modèles Prisma, diagrammes entité-relation, architecture SSE/Redis
BLOC 3 Préparation au déploiement	Préparer et documenter le déploiement, mettre en place des tests, gérer la sécurité.	Docker Compose, Vercel, tests d'intégration, jeux d'essai documentés
Transversal Sécurité & RGPD	Appliquer les bonnes pratiques de sécurité, identifier les risques, respecter le RGPD.	JWT + bcrypt, headers de sécurité, validation inputs, politique de cookies, minimisation des données

4. SPECIFICATIONS FONCTIONNELLES

4.1 ANALYSE DU BESOIN UTILISATEUR

L'application Konekt s'adresse à un public large souhaitant interagir avec son réseau en ligne. L'analyse du besoin a conduit à identifier quatre profils d'utilisateurs distincts avec des attentes spécifiques :

PROFIL	BESOINS PRINCIPAUX	FONCTIONNALITES ASSOCIEES
Utilisateur non connecté	Découvrir la plateforme, créer un compte, se connecter.	Page d'accueil publique, inscription, connexion JWT, OAuth Google
Utilisateur connecté (standard)	Partager du contenu, interagir avec son réseau, communiquer.	Posts, reactions, commentaires, stories, chat, friendship, notifications
Administrateur de groupe	Gérer les membres, modérer le contenu, organiser des événements	Panel d'administration groupe, gestion des rôles, RSVP événements
Développeur / mainteneur	Reproduire l'environnement, tester, déployer	Docker Compose, variables d'environnement, seed de données, tests automatisés

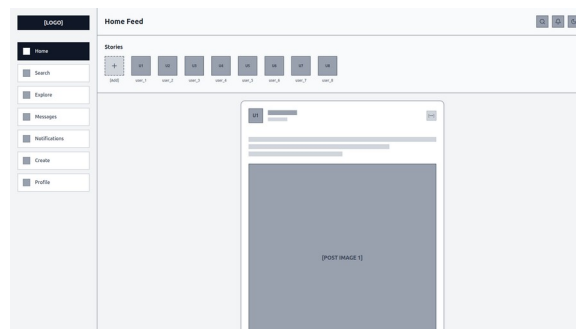
Illustrations de cadrage et maquettes fonctionnelles

Les écrans ci-dessous documentent le passage de l'analyse fonctionnelle vers des parcours utilisateur concrets : connexion, fil d'actualité, création de post, paramètres, stories et profil.

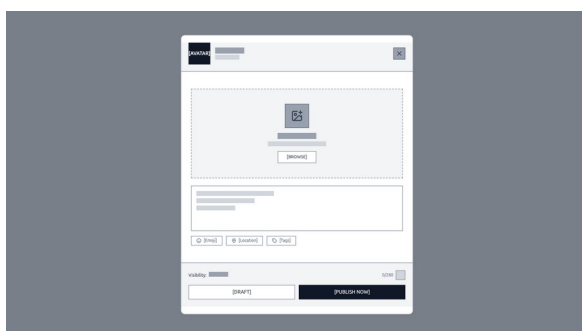
Ils servent de support de discussion pour valider l'ergonomie avant développement.



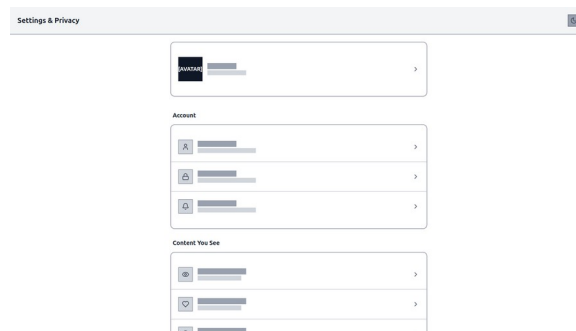
Wireframe - Connexion



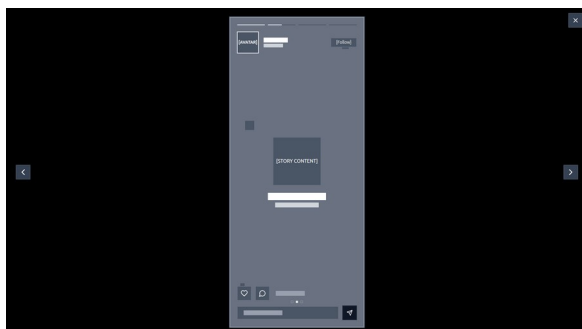
Wireframe - Fil d'actualité



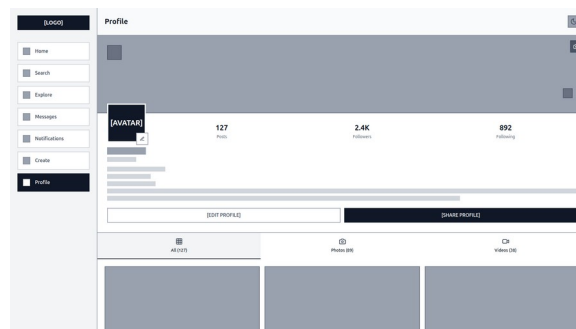
Wireframe - Création de publication



Wireframe - Paramètres et confidentialité



Wireframe – Consultation d'une story



Wireframe - Profil utilisateur

4.2 USER STORIES – SELECTION REPRESENTATIVE

Le projet couvre 38 user stories complètes (US001 à US038). Voici les principales user stories structurantes, avec leurs critères d'acceptation :

US001 — Inscription

AUTHENTIFICATION

"EN TANT QU'UTILISATEUR, JE VEUX CRÉER UN COMPTE AVEC EMAIL ET MOT DE PASSE POUR ACCÉDER À LA PLATEFORME."

Critères d'acceptation :

- Le formulaire valide le format email et le mot de passe (min. 8 caractères)
- Le mot de passe est hashé avec bcrypt avant stockage
- En cas d'email déjà utilisé, message d'erreur explicite

US004 — OAuth Google

AUTHENTIFICATION

"EN TANT QU'UTILISATEUR, JE VEUX ME CONNECTER AVEC MON COMPTE GOOGLE POUR ÉVITER LA GESTION D'UN MOT DE PASSE SUPPLÉMENTAIRE."

Critères d'acceptation :

- Le bouton Google OAuth redirige vers la page d'autorisation Google
- À la première connexion, un profil est créé automatiquement
- Le token JWT est émis avec la même durée de vie qu'une connexion classique

US010 — Fil d'actualité

POSTS

"EN TANT QU'UTILISATEUR CONNECTÉ, JE VEUX VOIR UN FIL D'ACTUALITÉ AVEC LES POSTS VISIBLES SELON MES RELATIONS."

Critères d'acceptation :

- Le fil affiche les posts PUBLIC de tous les comptes publics, les posts FRIENDS des comptes amis, et les posts des comptes privés si relation d'amitié acceptée
- Les posts sont triés par date décroissante
- La pagination est en offset (10 posts par page)
- Chaque post affiche les compteurs de réactions et commentaires

US018 — Réactions**INTERACTIONS**

"EN TANT QU'UTILISATEUR, JE VEUX RÉAGIR À UN POST AVEC L'UN DES 7 TYPES DE RÉACTIONS DISPONIBLES."

Critères d'acceptation :

- Les 7 réactions sont: Like, Dislike, Love, Laugh, Sad, Angry, Wow
- Un utilisateur ne peut avoir qu'une réaction par post (remplacée si rechargée)
- Le compteur se met à jour en temps quasi-réel (optimistic UI)
- La suppression d'une réaction est possible en re cliquant

US025 — Chat direct**MESSAGERIE**

"EN TANT QU'UTILISATEUR, JE VEUX ENVOYER DES MESSAGES PRIVÉS À UN AUTRE UTILISATEUR EN TEMPS RÉEL."

Critères d'acceptation :

- Les messages s'affichent immédiatement pour l'émetteur (optimistic update)
- Le destinataire reçoit le message via SSE sans recharger la page
- L'historique est paginé (50 messages par défaut)
- Le statut 'lu' est mis à jour lors de l'ouverture de la conversation

US031 — Notifications**NOTIFICATIONS**

"EN TANT QU'UTILISATEUR, JE VEUX VOIR LES COMPTEURS D'ACTIVITÉ LIÉS À MON COMPTE."

Critères d'acceptation :

- 3 compteurs affichés dans la navigation : invitations (amis + groupes), messages non lus, événements à venir
- Rafraîchissement automatique toutes les 30 secondes
- Badge numérique sur l'icône de navigation

BILAN USER STORIES

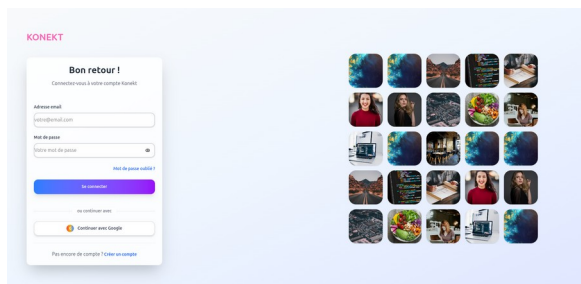
- 38 user stories complètes rédigées, couvrant les 8 domaines fonctionnels.
- Chaque US est associée à des critères d'acceptation mesurables.
- Priorisées par phases: MVP (Must Have), features (Should Have), Polish (corrections, optimisations et préparation soutenance).

4.3 FONCTIONNALITES PRINCIPALES

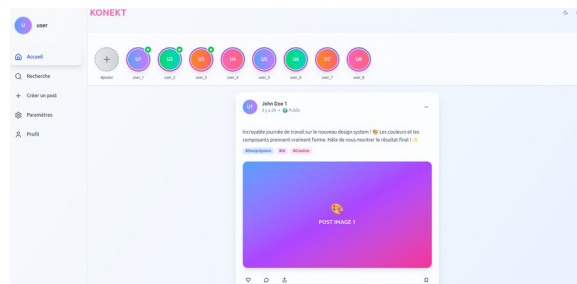
FONCTIONNALITE	DESCRIPTION	STACK TECHNIQUE
Authentification JWT	Connexion/inscription sécurisée, sessions stateless	jose, bcrypt, cookies httpOnly
OAuth Google	Connexion via compte Google	OAuth 2.0 (implémentation native)
Feed personnalisé	Fil d'actualité filtré par réseau social	Prisma, offset pagination(skip/take)
Posts & médias	Texte, images (upload), partage	Cloudinary, Prisma
Reactions (×7)	Like, Dislike, Love, Laugh, Sad, Angry, Wow	Prisma, optimistic UI
Stories 24h	Publications éphémères avec vue auto-expiration	Prisma, cron cleanup
Chat direct & groupe	Messagerie 1:1 et multi-utilisateurs	SSE, Redis Upstash
Groupes	Création, membres, rôles, posts de groupe	Prisma many-to-many
Evènements	Création, RSVP, liste participants	Prisma, composants React
Notifications (×12)	Temps réel, multi-types, compteur non-lus	SSE, Redis, Prisma
Recherche	Recherche utilisateurs et groupes	Prisma full-text search

Mockups validés

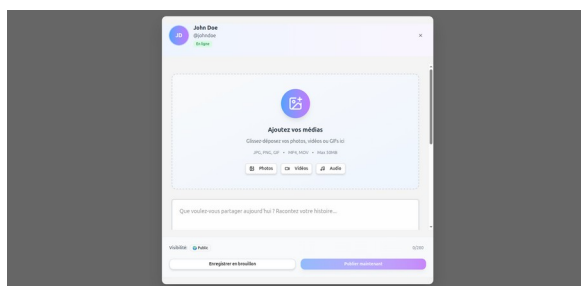
Les mockups matérialisent la direction graphique finale de Konekt : identité visuelle, hiérarchie des contenus, états interactifs et cohérence entre desktop, modales et vues immersives.



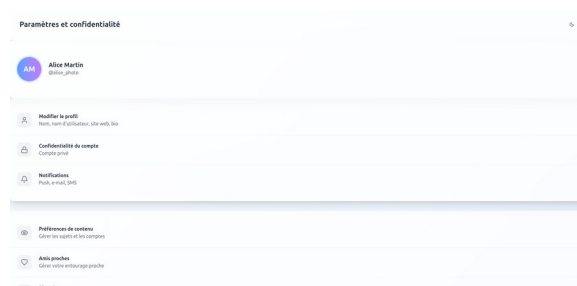
Mockup - Page de connexion



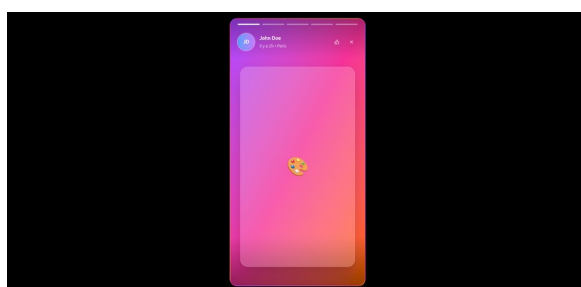
Mockup - Fil d'actualité



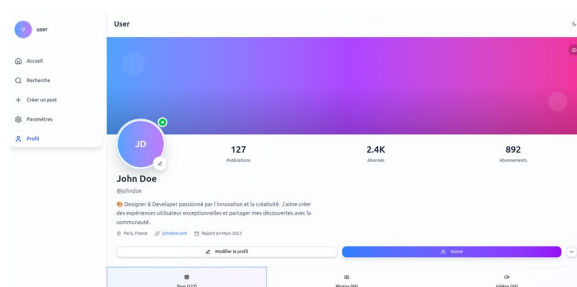
Mockup - Création de publication



Mockup - paramètres



Mockup - Story viewer



Mockup - Profil utilisateur

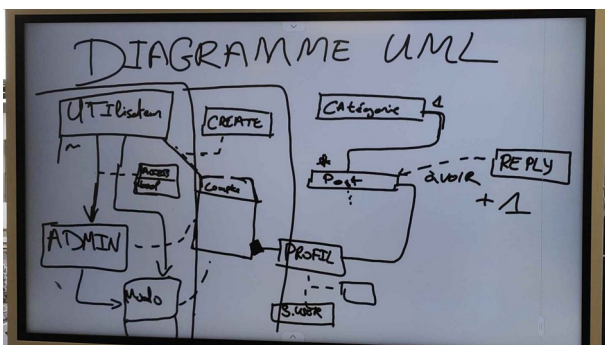
5. SPECIFICATIONS TECHNIQUES

5.0 CONCEPTION UML, MERISE ET SQL

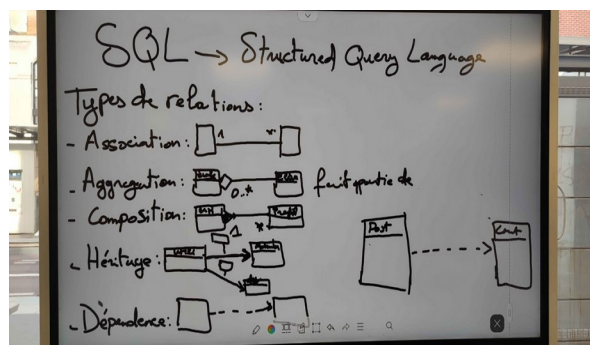
Avant l'implémentation, la modélisation a été travaillée en plusieurs passes : croquis UML en réunion, rappel des relations SQL, MCD Merise, puis traduction vers le schéma Prisma et PostgreSQL.

Cette chaîne de conception limite les angles morts : cardinalités, dépendances, ownership des données et contraintes métier. Le **dictionnaire de données complet** (18 modèles, types SQL, contraintes et cardinalités), via ce lien :

<https://github.com/CLecart/social-network/blob/docs/dossier-certification/dossier-certification/03-conception/donnees.md>

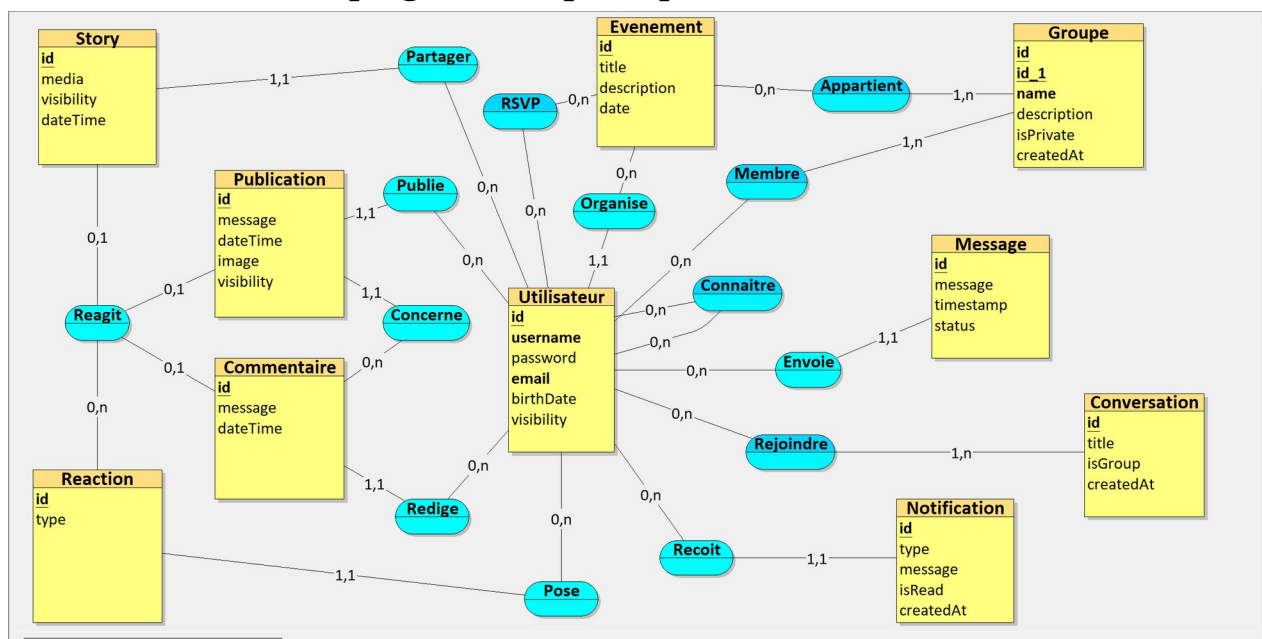


Atelier de conception - Diagramme UML et relations métier

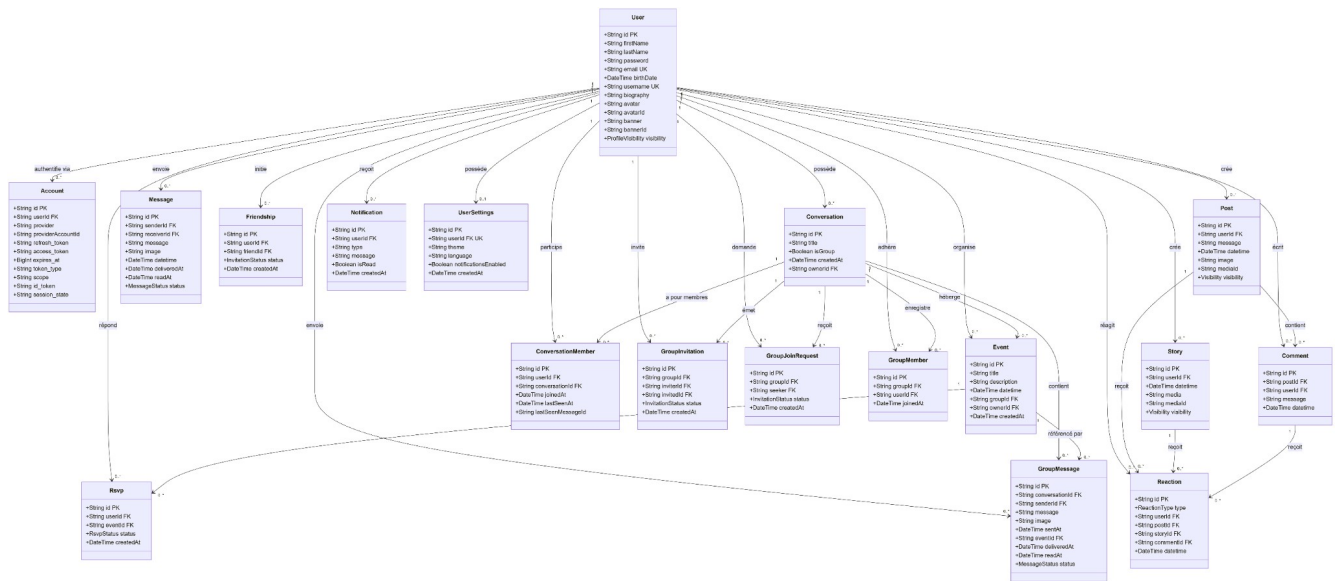


Atelier SQL - Types de relations et dépendances

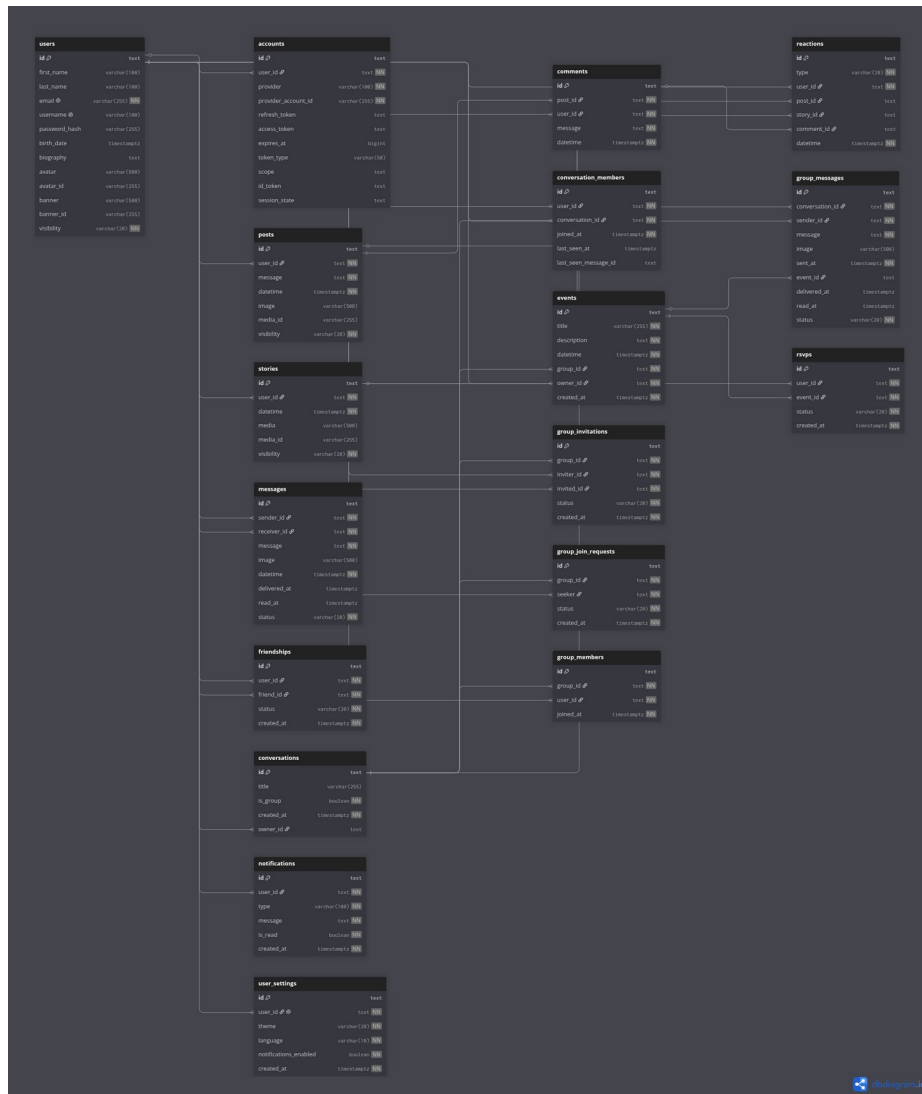
MCD Looping - entités principales et cardinalités Merise



MLD - Modèle Logique de Données (schéma relationnel intermédiaire)



MPD - Modèle Physique de Données (PostgreSQL) incluant les types SQL, clés primaires/étrangères et contrainte



Modélisation de Données

Dictionnaire de Données

Source : `prisma/schema.prisma` — traduit en types SQL PostgreSQL.

Conventions de types :

Type SQL	Description
<code>CHAR(25)</code>	Identifiant CUID (chaîne fixe 25 caractères)
<code>VARCHAR(255)</code>	Chaîne courte (nom, URL, token)
<code>TEXT</code>	Chaîne longue (message, biographie)
<code>TIMESTAMP</code>	Date et heure
<code>BOOLEAN</code>	Vrai / Faux
<code>BIGINT</code>	Entier 64 bits (timestamp OAuth)
<code>ENUM(...)</code>	Valeur parmi un ensemble fini

Modélisation relationnelle :

Cardinalité → 1,N entre User et Post

Un user écrit plusieurs posts, un post a exactement un auteur

Many-to-many → User ↔ Conversation via ConversationMember

Contrainte d'unicité → `@unique([userId, postId])` sur les réactions

Impossible de liker deux fois

Intégrité référentielle → `onDelete: Cascade`

Si un user est supprimé, ses données liées aussi

Clé primaire → `id` `CHAR(25)`

Identifiant unique CUID sur chaque table

Clé étrangère → `@relation(fields: [userId], references: [id])`

Partout

Model: User

Description : Entité centrale représentant un utilisateur du réseau social.

Attributs

Attribut	Type SQL	Contraintes	Description
<code>id</code>	<code>CHAR(25)</code>	PRIMARY KEY, NOT NULL	Identifiant CUID
<code>firstName</code>	<code>VARCHAR(255)</code>	NULL	Prénom
<code>lastName</code>	<code>VARCHAR(255)</code>	NULL	Nom de famille
<code>password</code>	<code>VARCHAR(255)</code>	NULL	Hash bcrypt du mot de passe
<code>email</code>	<code>VARCHAR(255)</code>	UNIQUE, NOT NULL	Adresse email
<code>birthDate</code>	<code>TIMESTAMP</code>	NULL	Date de naissance
<code>username</code>	<code>VARCHAR(50)</code>	UNIQUE, NULL	Pseudonyme
<code>biography</code>	<code>TEXT</code>	NULL	Biographie
<code>avatar</code>	<code>TEXT</code>	NULL	URL avatar (Cloudinary)
<code>avatarId</code>	<code>VARCHAR(255)</code>	NULL	ID asset Cloudinary
<code>banner</code>	<code>TEXT</code>	NULL	URL bannière (Cloudinary)
<code>bannerId</code>	<code>VARCHAR(255)</code>	NULL	ID asset Cloudinary
<code>visibility</code>	<code>ENUM('PUBLIC','PRIVATE')</code>	DEFAULT 'PUBLIC', NOT NULL	Visibilité du profil

Relations

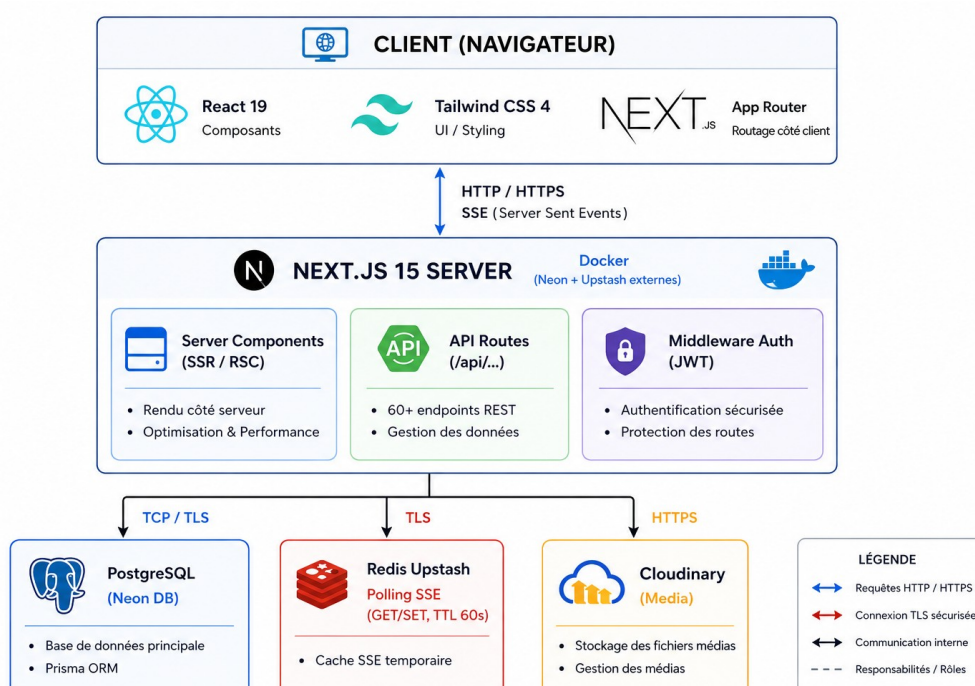
- `posts` : `Post[]` — Posts créés
- `comments` : `Comment[]` — Commentaires
- `friendships` : `Friendship[]` — Amitiés initiées
- `friendsWithMe` : `Friendship[]` — Amitiés reçues
- `conversations` : `Conversation[]` — Conversations créées
- `conversationMembers` : `ConversationMember[]` — Participations aux conversations
- `messages` : `Message[]` (SentMessages) — Messages envoyés
- `receivedMessages` : `Message[]` — Messages reçus
- `notifications` : `Notification[]` — Notifications reçues
- `reactions` : `Reaction[]` — Réactions publiées
- `stories` : `Story[]` — Stories publiées
- `rsvps` : `Rsvp[]` — Réponses aux événements
- `groupInvitations` : `GroupInvitation[]` (invitant + invité)
- `groupMembers` : `GroupMember[]` — Adhésions aux groupes
- `groupMessages` : `GroupMessage[]` — Messages de groupe
- `eventsCreated` : `Event[]` — Événements créés
- `userSettings` : `UserSettings` — Préférences
- `accounts` : `Account[]` — Comptes OAuth (Google)

5.1 ARCHITECTURE LOGICIELLE GLOBALE

Le projet repose sur une architecture full-stack Next.js 15 avec l'App Router, qui permet de cohabiter les React Server Components (rendus côté serveur) et les Client Components (interactifs) dans une même application.

COUCHE	RÔLE	TECHNOLOGIES
Interface (UI)	Rendu des pages, composants interactifs	React 19, Tailwind CSS, Next.js App Router
API / Logique métier	Endpoints REST, validation, business logic	Next.js API Routes, Zod, TypeScript
Accès aux données	ORM, migrations, requêtes optimisées	Prisma ORM, PostgreSQL (Neon)
Temps réel	Messagerie et notifications live	SSE (Server-Sent Events), Redis Upstash
Sécurité	Auth, tokens, middleware	JWT (jose), bcrypt, Next.js middleware
Médias	Upload, transformation, CDN	Cloudinary
Infrastructure	Conteneurisation, CI/CD	Docker, Vercel

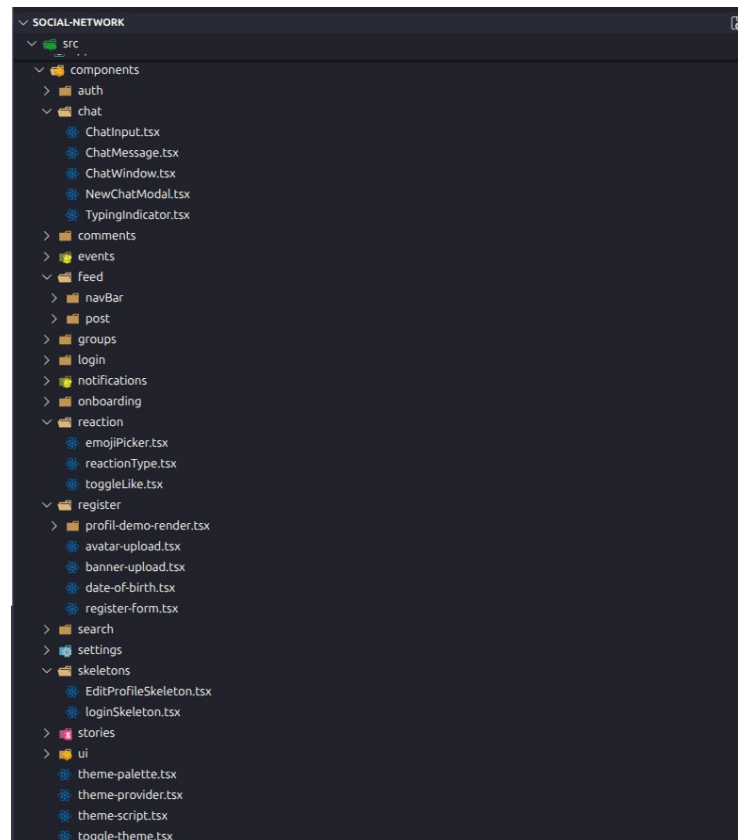
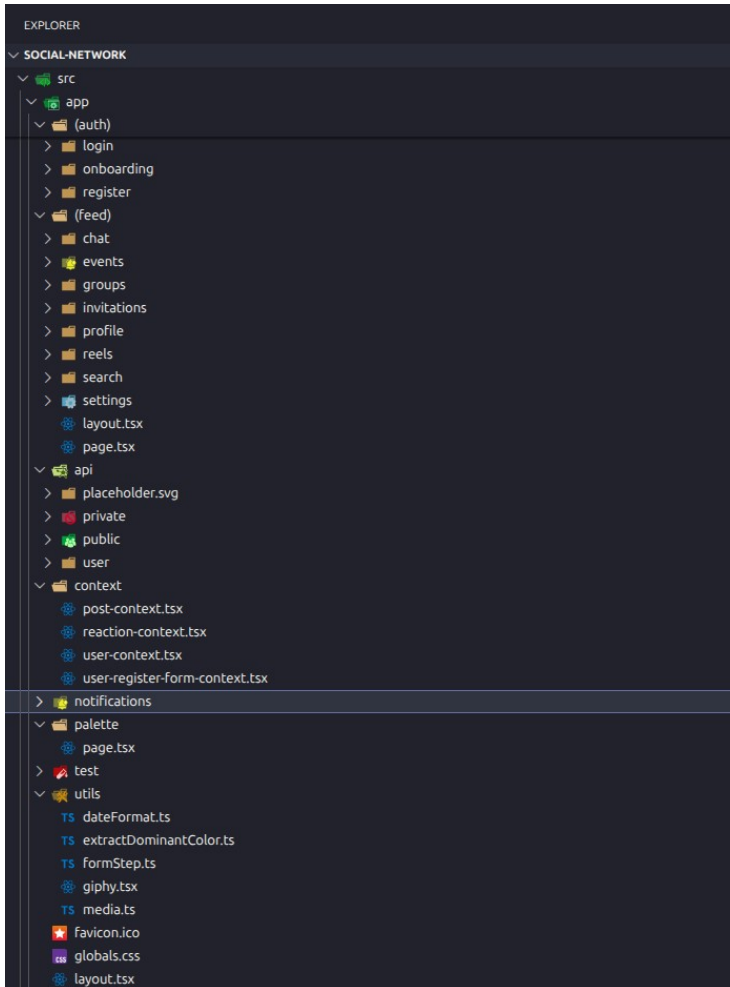
5.2 ARCHITECTURE LOGICIELLE SIMPLIFIEE

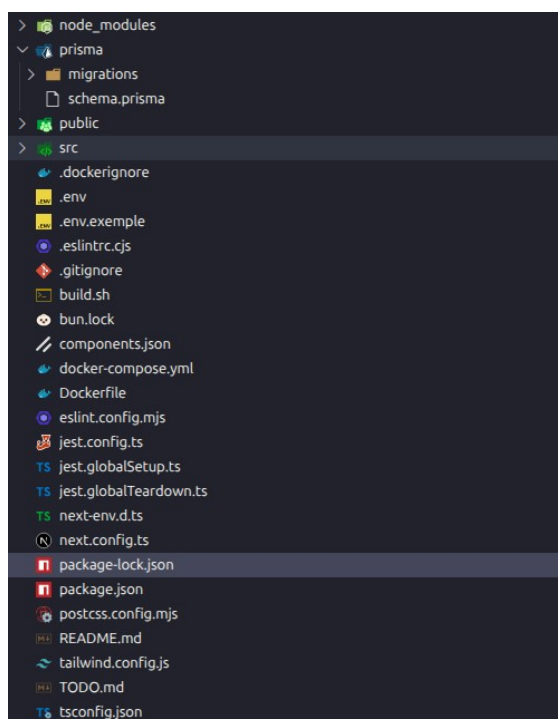
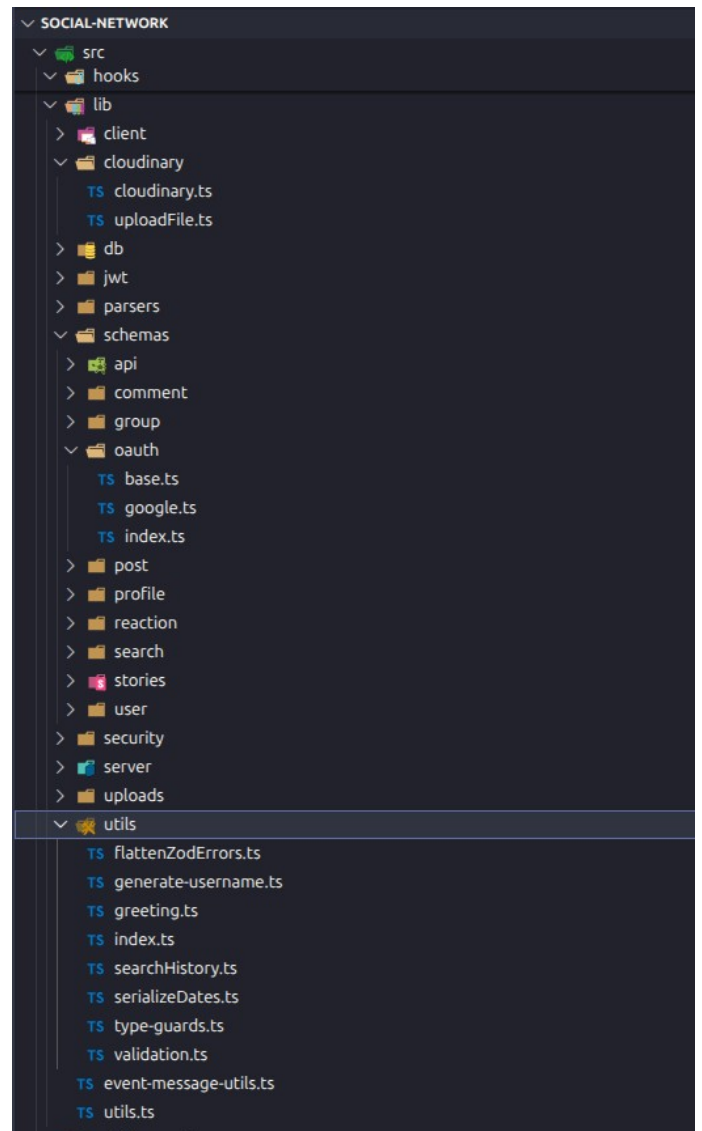
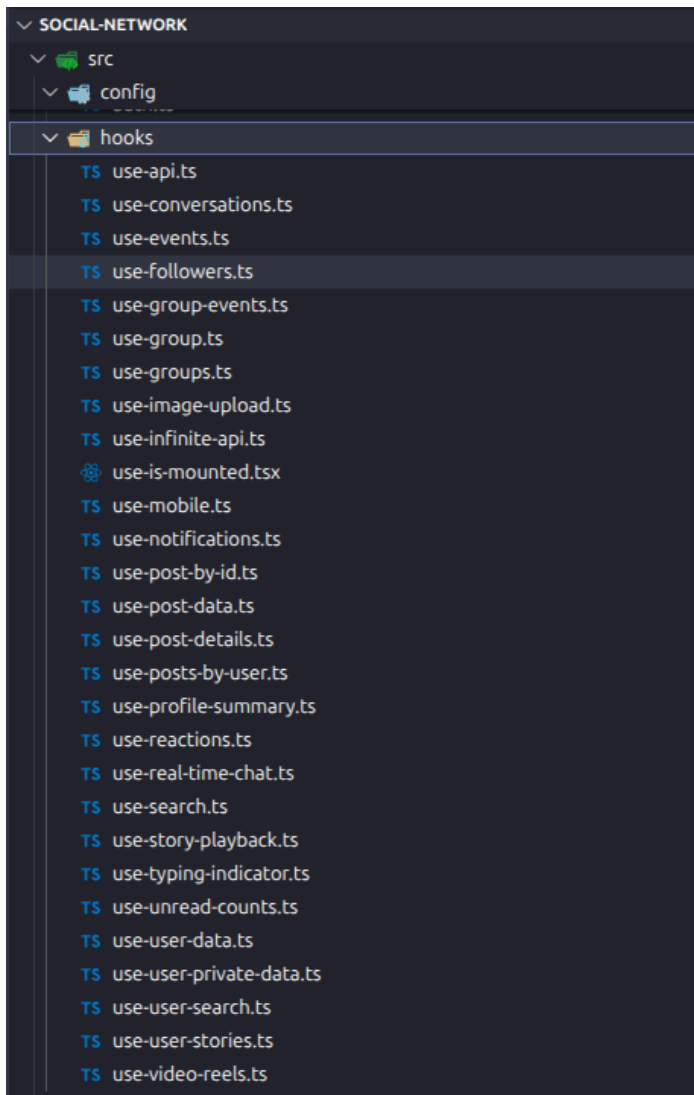


5.3 ORGANISATION DES FICHIERS – APP ROUTER

L'application suit la convention App Router de Next.js 15, avec une séparation stricte entre le code serveur et le code client.

Structure du projet





5.4 TECHNOLOGIES UTILISEES

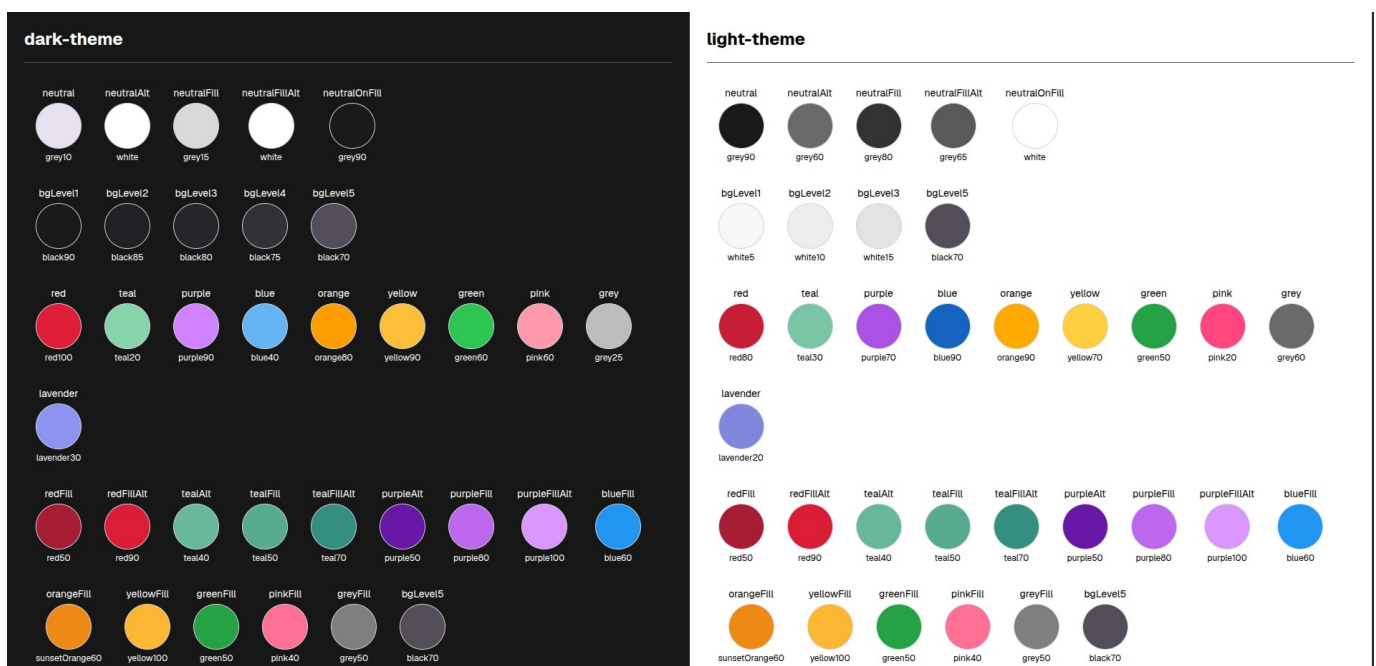
TECHNOLOGIE	VERSION	RÔLE	JUSTIFICATION
Next.js	15.x	Framework full-stack	App Router, RSC, API Routes intégrées
React	19.x	UI Components	Concurrent features, Server Components
TypeScript	5.x	Typage statique	Sécurité du code, refactoring, DX
Tailwind CSS	4.x	Design system	Utility-first, responsive, dark mode
Prisma ORM	6.x	Accès BDD	Type-safe, migrations, studio GUI
PostgreSQL	16	Base de données	Relationnel, ACID, support JSON
Neon DB	—	PostgreSQL serverless	Branching, connexion directe + pooler
Redis Upstash	1.x	Bus temps réel	Serverless Redis, pub/sub SSE
jose	6.x	JWT HS256	Léger, compatible Edge Runtime
bcrypt	6.x	Hashage mot de passe	Standard industrie, salt rounds 12
Cloudinary	2.x	CDN médias	Upload, transformation, optimisation auto
Zod	3.x	Validation des données	Schema-first, inférence TypeScript
Docker	—	Conteneurisation	Environnement reproductible dev/prod

Design system et thèmes

La palette ci-dessous formalise les tokens visuels du projet : couleurs neutres, niveaux de fond, variantes d'accent et déclinaisons light/dark.

Elle garantit une interface cohérente entre les composants Tailwind et les écrans finaux.

Palette light/dark theme - tokens de couleurs du design system



5.5 MODELE DE DONNEES – 18 ENTITES PRISMA

Le schéma de données est le cœur du projet. Il comprend 18 modèles Prisma interconnectés par plus de 40 relations. Voici les modèles principaux avec leurs attributs clés.

Modèles principaux

MODELE	CHAMPS PRINCIPAUX	RELATIONS
User	id, email, name, image, bio, createdAt	posts, stories, follows, friends, messages, notifications, groups
Post	id, content, mediaUrl, authorId, groupId, createdAt	author, comments, reactions, group, shares
Comment	id, content, postId, authorId, parentId	post, author, children (replies), reactions
Reaction	id, type (7 valeurs), userId, postId/commentId	user, post/comment (polymorphe)
Story	id, mediaUrl, authorId, expiresAt, viewCount	author, views
Message	id, content, senderId, conversationId, readAt	sender, conversation
Conversation	id, isGroup, name, participants	messages, participants (many-to-many)
Conversation (groupe)	id, title, isGroup (true), createdAt	members (ConversationMember), messages (Groupmessage), invitations
Event	id, title, date, location, organizerId, groupId	organizer, group, attendees (RSVP)
Notification	id, type (12 valeurs), userId, read, relatedId	user, related entity
Friendship	id, userId, friendId, status, createdAt	user, friend (User) – statut <i>Pending/Accepted/Declined</i>

Extrait du schéma Prisma – Modèle « Post »

```
model Post {
  id          String      @id @default(cuid())
  userId      String
  message     String
  datetime    DateTime    @default(now())
  image       String?
  visibility  Visibility  @default(PUBLIC)
  mediaId     String?
  comments    Comment[]
  user        User        @relation(fields: [userId], references: [id])
  reactions   Reaction[]
}
```

Gabriel K, 11 months ago via PR #71 · refactor(onboarding): improve user info types, ...

Extrait – Modèle « Message et Conversation »

```
model Message {
  id          String      @id @default(cuid())
  senderId    String
  receiverId   String
  message     String
  image       String?
  datetime    DateTime    @default(now())
  deliveredAt  DateTime?
  readAt      DateTime?
  status      MessageStatus @default(SENT)
  receiver    User        @relation("ReceivedMessages", fields: [receiverId], references: [id])
  sender      User        @relation("SentMessages", fields: [senderId], references: [id])
}

model Friendship {
  id          String      @id @default(cuid())
  userId      String
  friendId    String
  status      InvitationStatus
  createdAt   DateTime    @default(now())
  friend      User        @relation("FriendOf", fields: [friendId], references: [id])
  user        User        @relation("UserFriendships", fields: [userId], references: [id])
}
@@unique([userId, friendId])

model Conversation {
  id          String      @id @default(cuid())
  title       String?
  isGroup     Boolean      @default(false)
  createdAt   DateTime    @default(now())
  ownerId     String?
  User        User?        @relation(fields: [ownerId], references: [id])
  members     ConversationMember[]
  events      Event[]
  GroupInvitation GroupInvitation[]
  GroupJoinRequest GroupJoinRequest[]
  GroupMember  GroupMember[]
  messages     GroupMessage[]
}
```

Gabriel K, 8 months ago via PR #118 · chore(prisma):

Extrait – Modèle « Type de Réactions »

```
model Reaction {
  id          String      @id @default(cuid())
  type        ReactionType // LIKE | DISLIKE | LOVE | LAUGH | SAD | ANGRY | WOW
  userId      String
  postId      String?
  datetime    DateTime    @default(now())
  storyId     String?
  commentId   String?
  comment     Comment?    @relation(fields: [commentId], references: [id])
  post        Post?       @relation(fields: [postId], references: [id])
  story       Story?      @relation(fields: [storyId], references: [id])
  user        User        @relation(fields: [userId], references: [id])

  @@unique([userId, postId])
  @@unique([userId, storyId])
  @@unique([userId, commentId])
}
```

clecart, 9 months ago · pull db prisma

Extrait – Modèle « Notification »

```
model Notification {
  id          String      @id @default(cuid())
  userId      String
  type        String
  message     String
  isRead      Boolean     @default(false)
  createdAt   DateTime    @default(now())
  user        User        @relation(fields: [userId], references: [id])

  @@index([userId])
}

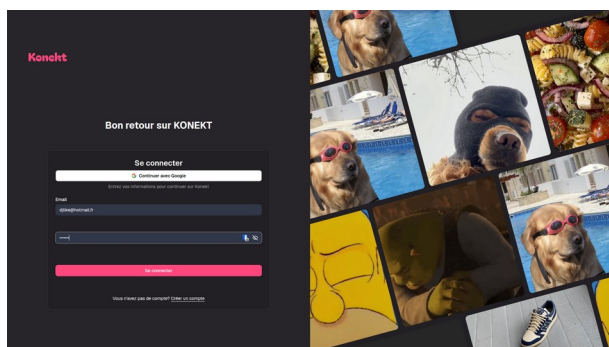
model UserSettings {
  id          String      @id @default(cuid())
  userId      String      @unique
  theme       String      @default("light")
  language    String      @default("en")
  notificationsEnabled Boolean @default(true)
  createdAt   DateTime    @default(now())
  user        User        @relation(fields: [userId], references: [id])
}
```

Gabriel K, 11 months ago via PR #71 · feat: improve onboarding page design and add en...

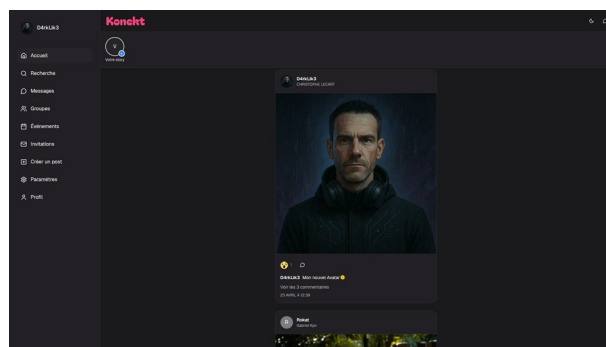
6. REALISATIONS TECHNIQUES

6.0 ECRANS IMPLEMENTES

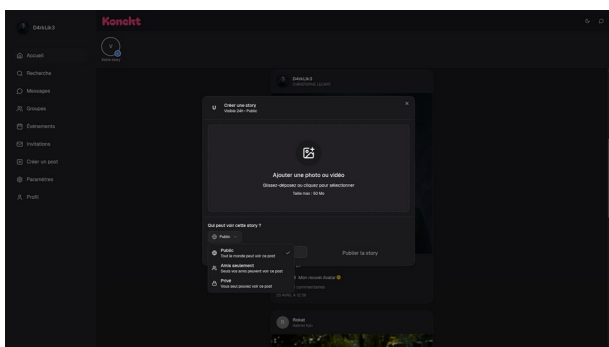
Les écrans suivants illustrent l'application telle qu'elle a été développée et livrée.



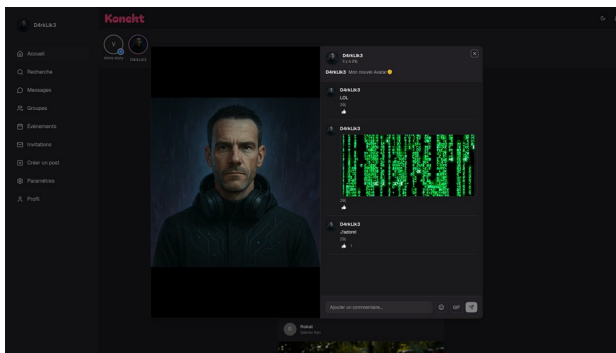
KONEKT – PAGE DE CONNECTION



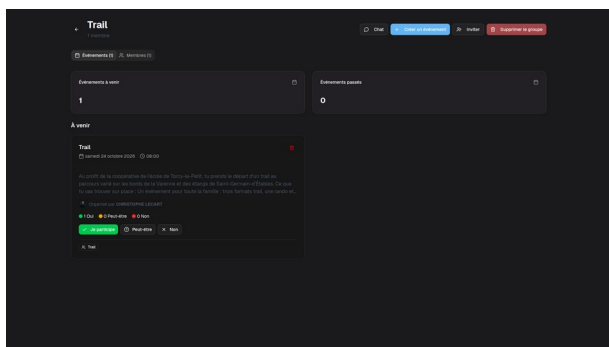
KONEKT – FIL D'ACTUALITES



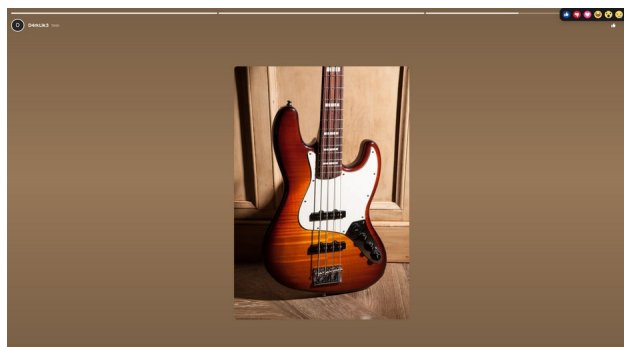
KONEKT – CREATION DE PUBLICATIONS



KONEKT - COMMENTAIRES



KONEKT – EVENEMENTS



KONEKT - STORY

6.1 AUTHENTICATION JWT + OAUTH GOOGLE

La sécurisation des accès est un enjeu central. Nous avons implémenté un système d'authentification en deux volets : connexion classique email/mot de passe avec tokens JWT, et connexion OAuth Google.

Flux d'authentification JWT

1. L'utilisateur soumet email + password.
2. Le serveur vérifie l'email en base (Prisma) → 401 si inexistant.
3. `bcrypt.compare(password, hash)` → 401 si incorrect.
4. Génération token JWT (`jose`, HS256, exp: 480min / 8h).
5. Token stocké en cookie `httpOnly`, `Secure`, `SameSite=Lax`.
6. Middleware Next.js vérifie le token sur chaque requête protégée.

Implémentation — Middleware de protection des routes

```

TS route.ts x
src > app > api > public > auth > login > TS route.ts > POST
You, 2 days ago | 2 authors (Gabriel K and one other)
1 import { login } from "@lib/server/user/login";
2 import { checkRateLimit } from "@lib/security/ratelimit";
3 import { NextRequest, NextResponse } from "next/server";
4
5 export async function POST(req: NextRequest): Promise<NextResponse> {
6   const rateLimitResponse = await checkRateLimit(req);
7   if (rateLimitResponse) return rateLimitResponse;
8
9   try {
10    const { email, password } = await req.json();
11    const token = await login(email, password);
12
13    const res = NextResponse.json({ success: true });
14    res.cookies.set("authToken", token, {
15      httpOnly: true,
16      secure: process.env.NODE_ENV === "production",
17      sameSite: "lax",
18      maxAge: 60 * 60 * 24 * 7,
19      path: "/",
20    });
21
22    return res;
23  } catch (error) {
24    console.error("Login error:", error);
25    if (error instanceof Error) {
26      if (error.message.toLowerCase().includes("invalid")) {
27        return NextResponse.json({ errors: { general: "Invalid email or password" } }, { status: 401 });
28      }
29    }
30    return NextResponse.json(
31      { error: "Internal server error" },
32      { status: 500 }
33    );
34  }
35 }

```

6.2 API REST – 60+ ENDPOINTS

Les routes API sont organisées en deux zones : `/api/public/` (authentification, inscription) et `/api/private/` (toutes les fonctionnalités métier, protégées par le middleware JWT). Chaque endpoint valide les entrées via Zod et retourne une réponse standardisée.

Exemple — Endpoint fil d'actualité avec pagination offset (skip/take)

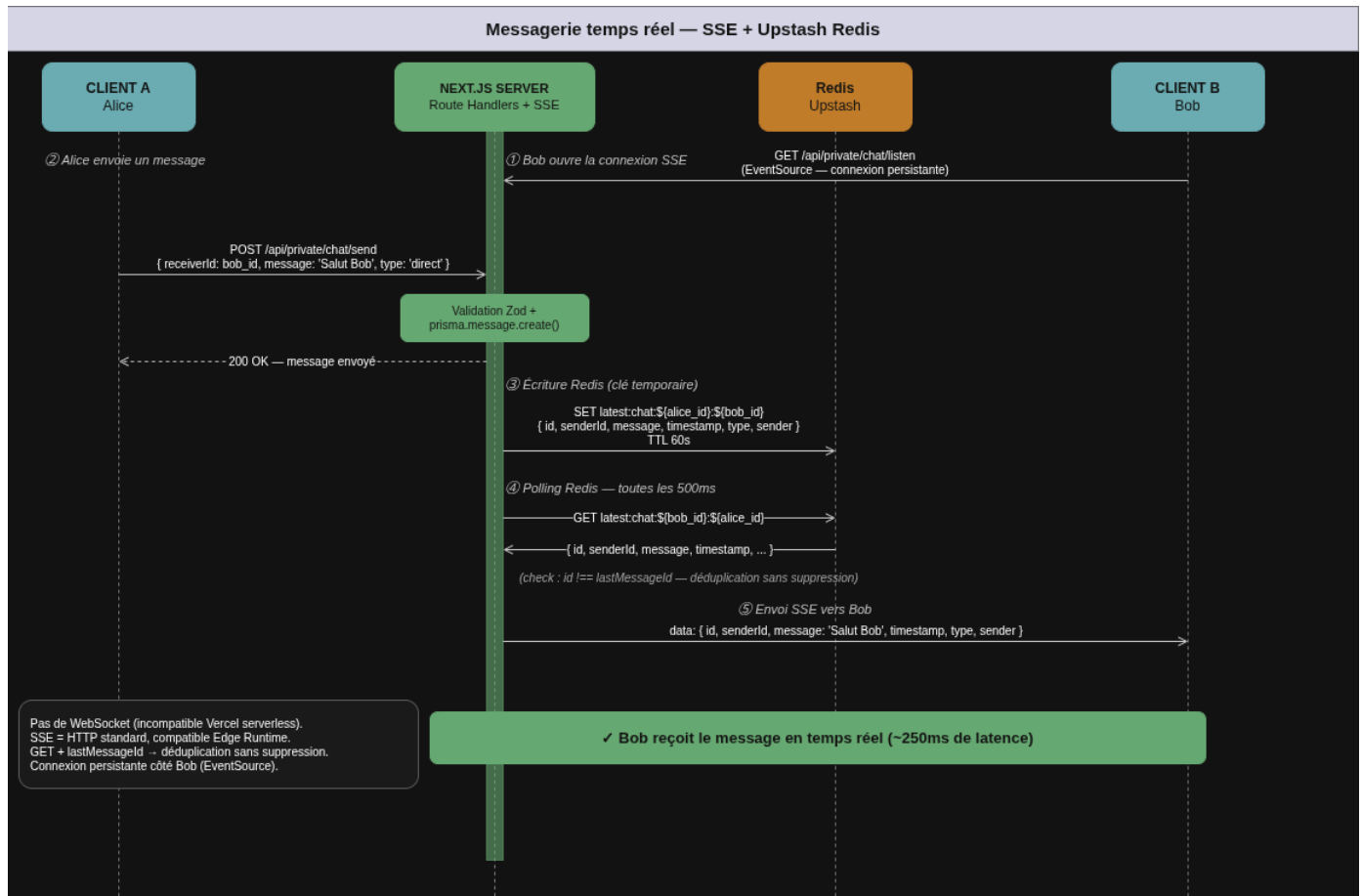
```

TS route.ts x
src > app > api > private > post > getAllPosts > TS route.ts > GET
Gabriel K, 8 months ago | 3 authors (clecart and others)
1 import { getPaginatedPosts } from "@lib/db/queries/post/getAllPosts";
2 import { respondError, respondSuccess } from "@lib/server/api/response";
3 import { NextRequest, NextResponse } from "next/server";
4 import { getUserIdFromRequest } from "@lib/server/api/getUserId";
5
6 export async function GET(req: NextRequest) {
7   try {
8     const userId = await getUserIdFromRequest(req);
9
10    if (!userId) {
11      return NextResponse.json(respondError("Missing or invalid user ID"), {
12        status: 401,
13      });
14    }
15
16    const { searchParams } = new URL(req.url);
17
18    const skip = parseInt(searchParams.get("skip") || "0", 10);
19    const take = parseInt(searchParams.get("take") || "10", 10);
20
21    if (isNaN(skip) || skip < 0 || isNaN(take) || take ≤ 0 || take > 50) {
22      return NextResponse.json(respondError("Invalid pagination parameters."), {
23        status: 400,
24      });
25    }
26
27    const posts = await getPaginatedPosts(skip, take, userId);
28
29    return NextResponse.json(
30      respondSuccess(
31        posts,
32        posts.length === 0 ? "No posts available yet." : undefined
33      ),
34      { status: 200 }
35    );
36  } catch (error) {
37    console.error("❌ Failed to fetch posts:", error);
38
39    return NextResponse.json(
40      respondError(
41        error instanceof Error ? error.message : "Unexpected server error."
42      ),
43      { status: 500 }
44    );
45  }
46 }
  
```

6.3 MESSAGERIE TEMPS REEL – SSE+REDIS

La messagerie est la fonctionnalité la plus complexe du projet, car elle nécessite un mécanisme de push en temps réel sans WebSocket (incompatible avec Vercel serverless). Nous avons implémenté un système basé sur Server-Sent Events (SSE) côté transport et Redis Upstash comme bus de messages.

Architecture SSE + Redis



Implémentation — Endpoint SSE stream

```

Ts route.ts x
src > app > api > private > chat > listen > Ts route.ts > GET > stream > start > pollForMessages

18 export async function GET(request: NextRequest) {
23   const stream = new ReadableStream({
24     start(controller) {
33       // This is a simplified SSE polling approach - for production, consider using pub/sub or a managed real-time service
34
35       let isActive = true;
36       let lastMessageId = "";
37
38       const pollForMessages = async () => {
39         while (isActive) {
40           try {
41             // Check Redis for new messages
42             const channels =
43               type === "direct"
44                 ? [
45                   `latest:chat:${userId}:${conversationId}`,
46                   `latest:chat:${conversationId}:${userId}`,
47                 ]
48                 : [`latest:chat:group:${conversationId}`];
49
50             for (const channel of channels) {
51               const messageData = await redisdb.get(channel);
52               if (
53                 messageData &&
54                 typeof messageData === "object" &&
55                 "id" in messageData
56               ) {
57                 const typedMessageData = messageData as MessageData;
58                 // Check if this is a new message
59                 if (typedMessageData.id !== lastMessageId) {
60                   lastMessageId = typedMessageData.id;
61
62                   // Send the message to the client
63                   controller.enqueue(...
64                     );
65                 }
66               }
67             }
68           } catch (error) {
69             // pcaboor, 9 months ago • Private message / notifications / Events / Grou...
70             // Check for status updates for this user
71             const statusUpdateKeys = await redisdb.keys(
72               `status_update:${userId}:*`,
73             );
74             for (const key of statusUpdateKeys) {
75               const statusUpdate = await redisdb.get(key);
76               if (statusUpdate && typeof statusUpdate === "object") {
77                 // Send status update to client
78                 controller.enqueue(...
79                   );
80               }
81
82               // Remove the status update after sending
83               await redisdb.del(key);
84             }
85           }
86         } catch (error) {
87           //
88         }
89
90         // Wait 500ms before polling again for faster updates
91         await new Promise((resolve) => setTimeout(resolve, 500));
92       }
93     };
94
95     // Start polling
96     pollForMessages();
97
98     // Handle client disconnect
99     request.signal.addEventListener("abort", () => {
100       isActive = false;
101       controller.close();
102     });
103   },
104   );
105 }

```

6.4 SYSTEMES DE REACTIONS UNIVERSELLES

Le système de réactions permet aux utilisateurs d'exprimer 7 types d'émotions sur n'importe quel post ou commentaire. Il repose sur un modèle Prisma avec contrainte d'unicité (un utilisateur = une réaction par entité) sur n'importe quel *POST*, *STORY* OU *COMMENTAIRE*.

```

TS route.ts x
src > app > api > private > reaction > TS route.ts > PUT
Gabriel K, 8 months ago | 2 authors (Gabriel K and one other)
1 import { updatedReaction } from "@lib/db/queries/reaction/updatedReaction";
2 import { CreateReaction, ReactionSchemas } from "@lib/schemas/reaction";
3 import { respondError, respondSuccess } from "@lib/server/api/response";
4 import { parseOrThrow, ValidationError } from "@lib/utils/validation";
5 import { NextRequest, NextResponse } from "next/server";
6 import { getUserIdFromRequest } from "@lib/server/api/getUserId";
7
8 export async function PUT(req: NextRequest) {
9   const userId = await getUserIdFromRequest(req);
10
11   if (!userId) {
12     return NextResponse.json(respondError("Invalid user ID"), { status: 401 });
13   }
14
15   let parsedData: CreateReaction;
16   try {
17     parsedData = parseOrThrow(ReactionSchemas.Create, await req.json(), { label: 'CreateReactionBody' });
18   } catch (error) {
19     if (error instanceof ValidationError) {
20       return NextResponse.json(
21         respondError("Invalid body", error.fieldErrors),
22         { status: 400 }
23       );
24     }
25     return NextResponse.json(
26       respondError(
27         error instanceof Error ? error.message : "Unexpected server error."
28       ),
29       { status: 500 }
30     );
31   }
32   try {
33     await updatedReaction(userId, parsedData);
34
35     return NextResponse.json(respondSuccess(null), { status: 200 });
36   } catch (error) {
37     console.error("❌ Failed to updated reaction:", error);
38
39     return NextResponse.json(
40       respondError(
41         error instanceof Error ? error.message : "Unexpected server error."
42       ),
43       { status: 500 }
44     );
45   }
46 }

```

Exemples de commit utilisé

 **clecart** · 10 months ago (July 8th, 2025 10:42 AM)

🌟 feat: Universal likes system implementation - Part 1

🎯 Core Like Component Features:

- Implement LikeComponent supporting stories, posts, and comments
- Add flying heart animations with proper cleanup and unique keys
- Create adaptive styling based on contentType (white for stories, CSS vars for posts/comments)
- Add real-time like count updates with optimistic UI

🔗 API Integration:

- Connect to updatedReaction endpoint with proper contentType mapping
- Support all three content types: 'stories', 'post', 'comment'
- Add comprehensive error handling and logging
- Implement proper mediaId validation and type checking

👤 UI/UX Improvements:

- Heart button turns red when liked with smooth transitions
- Flying hearts appear on like action with 1s animation
- Like count displays only when > 0 with dynamic updates
- Responsive styling that adapts to story viewer vs post/comment contexts

🔧 Backend Support:

- Update updatedReaction function to handle all three content types
- Add proper existence validation for posts, stories, and comments
- Implement unified cache logic and data creation patterns

🔥 Code Quality:

- Add proper TypeScript types for all content variations
- Implement consistent logging for debugging
- Use proper React keys to prevent conflicts
- Add comprehensive error boundaries and fallbacks

📦 Component Integration:

- Integrate LikeComponent in story-viewer, postCard, and postDetails
- Remove GIF functionality from post creation to avoid confusion
- Fix placeholder image 404 errors with proper fallbacks

🚧 TODO - Part 2:

- Fix cache persistence for comments and posts (currently only works for stories)
- Improve cache key logic in useEffect and initial state
- Add proper cache initialization across all content types
- Optimize component re-renders and state management

🔗 158d8ee | 🔍 | 🌐 | 📄 | 📁 | 📂 | 📅 | 📆 | 📇 | 📈 | 📉 | 📊 | 📋 | 📌 | 📍 | 📎 | 📏 | 📐 | 📑 | 📒 | 📓 | 📔 | 📕 | 📖 | 📗 | 📘 | 📙 | 📚 | 📛 | 📜 | 📝 | 📞 | 📟 | 📠 | 📡 | 📢 | 📣 | 📤 | 📥 | 📦 | 📧 | 📨 | 📩 | 📪 | 📫 | 📬 | 📭 | 📮 | 📯 | 📰 | 📱 | 📲 | 📳 | 📴 | 📵 | 📶 | 📷 | 📸 | 📹 | 📺 | 📻 | 📼 | 📽 | 📾 | 📿 | 📰 | 📱 | 📲 | 📳 | 📴 | 📵 | 📶 | 📷 | 📸 | 📹 | 📺 | 📻 | 📼 | 📽 | 📾 | 📿

+);

Changes 🔗 e7504ea ↔ 🔗 158d8ee | 🔍

6.5 GROUPES, EVENEMENTS ET NOTIFICATIONS

Les groupes sont une fonctionnalité structurante qui met en jeu des relations complexes: un utilisateur peut créer un groupe, inviter d'autres membres et organiser des événements. Les événements sont liés à des groupes et permettent une gestion des RSVP.

Modèle de groupe — Gestion des rôles

```
const isMember = group.members.some(member => member.userId === userId);
const isOwner = group.ownerId === userId;
pcaboar, 9 months ago • Private message / notifications / Events / Grou...
if (!isMember && !isOwner) {
  return NextResponse.json(respondError('Not authorized to add members'), { status: 403 });
}
```

```
const isOwner = group.ownerId === userId;
const isRemovingSelf = memberIdToRemove === userId;
pcaboar, 9 months ago • Private message / notifications / Events / Grou...
if (!isOwner && !isRemovingSelf) {
  return NextResponse.json(respondError('Not authorized to remove this member'), { status: 403 });
}
```

Système de notifications — 2 types

Les notifications sont créées directement dans les fonctions métier concernées. Le modèle Prisma utilise un champ type String libre, ce qui rend l'architecture extensible sans migration. Deux types sont implémentés : GROUP_INVITATION (invitation à rejoindre un groupe) et message_status_update (mise à jour du statut d'un message lu ou livré).

TYPE	DECLENCHEUR
group_invitation	Invitation à rejoindre un groupe
message_status_update	Mise à jour statut message (lu/livré)

TS createGroupInvitation.ts X

src > lib > db > queries > groups > TS createGroupInvitation.ts > createGroupInvitation

Zanakan12, 10 months ago | 1 author (Zanakan12)

```

1  import { db } from "@lib/db";
2
3  Zanakan12, 10 months ago | 1 author (Zanakan12)
4  interface CreateGroupInvitation {
5      groupId: string;
6      inviterId: string;
7      invitedId: string;
8  }
9
10 export async function createGroupInvitation(data: CreateGroupInvitation) {
11     const { groupId, inviterId, invitedId } = data;
12
13     const group = await db.conversation.findUnique({
14         where: { id: groupId },
15     });
16
17     if (!group) {
18         throw new Error("Group not found.");
19     }
20
21     if (group.ownerId !== inviterId) {
22         throw new Error("Not authorized to invite in this group.");
23     }
24
25     const invitation = await db.groupInvitation.create({
26         data: {
27             groupId,
28             inviterId,
29             invitedId,
30         },
31     });
32
33     Zanakan12, 10 months ago · refactor and moove groups to private route and ...
34     await db.notification.create({
35         data: {
36             userId: invitedId,
37             type: "GROUP_INVITATION",
38             message: `You have been invited to join the group "${
39                 group.title || "Unnamed Group"
40             }".`,
41         },
42     });
43
44     return invitation;
45 }

```


6.6 PERFORMANCES – OPTIMISATION DES REQUETES PRISMA

Les requêtes Prisma ont été optimisées pour éviter le problème N+1. Le fil d'actualité charge les posts, leurs réactions et leurs compteurs en une seule requête SQL grâce aux **include** et **_count**.

```
TS getAllPosts.ts M ●
src > lib > db > queries > post > TS getAllPosts.ts > ...
Gabriel K, 8 months ago | 5 authors (Gabriel K and others)
1 import { InvitationStatus } from "@prisma/client";
2 import { db } from "../..";
3 import { buildPostVisibilityFilter } from "../visibilityFilters";
4
5 export async function getPaginatedPosts(
6   skip: number,
7   take: number = 10,
8   currentUserId?: string,
9 ) {
10   const visibilityFilter = buildPostVisibilityFilter({
11     currentUserId,
12     showPrivatePosts: false,
13   });
14
15   return await db.post.findMany({
16     skip,
17     take,
18     where: visibilityFilter,
19     orderBy: { datetime: "desc" },
20     include: {
21       user: {
22         select: {
23           id: true,
24           username: true,
25           avatar: true,
26           lastName: true,
27           firstName: true,
28         },
29       },
30       reactions: {
31         select: {
32           id: true,
33           type: true,
34           user: {
35             select: {
36               id: true,
37               username: true,
38             },
39           },
40         },
41         _count: { select: { reactions: true, comments: true } },
42       },
43     },
44   });
45 }
```

pcaboer, 9 months ago • feat: gestion des visibilitées, messagerie, grou...

7. TESTS ET VALIDATION

7.1 STRATEGIE DE TEST

Nous avons mis en place une stratégie de test articulée autour de trois niveaux, en priorisant les tests d'intégration sur les endpoints API car ce sont eux qui offrent le meilleur rapport couverture/coût de maintenance.

NIVEAU	APPROCHE	OUTILS	COUVERTURE
Tests unitaires	Composants UI — comportements du Button	Jest, Testing Library	5 tests
Tests d'intégration	Endpoints auth avec BDD réelle	Jest, Next.js handlers	2 tests (register, login)
Tests manuels	Jeux d'essai documentés, simulation d'erreurs	—	45 cas de test

7.2 JEUX D'ESSAI - AUTHENTIFICATION

Voici la série complète des jeux d'essai pour le domaine Authentification (série AU) :

ID	CAS DE TEST	ENTRÉE SIMULEE	RESULTAT ATTENDU	✓
AU-01	Inscription valide	email valide, pwd ≥6 chars	HTTP 200, token JWT émis	✓
AU-02	Email déjà utilisé	email existant en BDD	HTTP 400, message erreur	✓
AU-03	Password trop court	pwd = '123'	HTTP 400, validation Zod	✓
AU-04	Connexion valide	credentials corrects	HTTP 200, cookie httpOnly	✓
AU-05	Mauvais mot de passe	email OK, pwd KO	HTTP 401, 'Invalid credentials'	✓
AU-06	Email inexistant	email inconnu	HTTP 401, 'Invalid credentials'	✓
AU-07	Token expiré	JWT exp dépassé	HTTP 401, redirect /login	✓
AU-08	Token falsifié	JWT signature KO	HTTP 401, middleware refuse	✓
AU-09	Accès route protégée sans token	Pas de cookie	Redirect /login	✓
AU-10	Déconnexion	POST /api/public/auth/logout	Cookie supprimé, HTTP 200	✓
AU-11	OAuth Google — nouveau compte	Token Google valide	Profil créé, JWT émis	✓

7.3 JEUX D'ESSAI – MESSAGERIE ET TEMPS REEL

ID	CAS DE TEST	ENTRÉE SIMULEE	RESULTAT ATTENDU	✓
CH-01	Envoi message 1:1	POST /api/private/chat/send	HTTP 200, message en BDD, SSE emit	✓
CH-02	Réception SSE	Client abonné au stream	Event reçu <1s après envoi	✓
CH-03	Message vide	message: "	HTTP 400, validation Zod	✓
CH-04	Conversation inexistante	conversationId: 'fake'	HTTP 404	✓
CH-05	Pagination messages	GET ?skip=0&take=30	30 messages retournés	✓
CH-06	Statut lu	PUT /api/private/messages/[id]/status body { status: "READ" }	readAt mis à jour en BDD	✓
CH-07	Chat de groupe	POST vers conv isGroup=true	Tous les membres notifiés via SSE	✓
CH-08	Déconnexion SSE	Client ferme la connexion	Ressource Redis libérée, pas d'erreur	✓

7.4 JEUX D'ESSAI – SECURITE

ID	CAS DE TEST	ATTAQUE SIMILEE	COMPORTEMENT ATTENDU	✓
SE-01	Injection SQL via Prisma	Paramètre avec ; DROP TABLE --	Prisma paramétrise → aucun effet	✓
SE-02	XSS dans un post	message: 'alert(1)'	Échappement HTML automatique	✓
SE-03	Accès données autre user	GET /api/users/[otherId]/private	HTTP 403 Forbidden	✓
SE-04	Brute force connexion	100 requêtes POST /login	Rate limiter déclenché	✓
SE-05	Upload fichier malveillant	Fichier .exe via Cloudinary	Rejet type MIME non autorisé	✓
SE-06	Token dans URL (leak)	JWT passé en query string	Refusé, cookie only accepté	✓

7.5 JEUX D'ESSAI – POSTS ET INTERACTIONS

ID	CAS DE TEST	ENTRÉE SIMULEE	RESULTAT ATTENDU	✓
PO-01	Créer un post texte	POST /api/private/post, message valide	HTTP 201, post en BDD	✓
PO-02	Post avec image	POST avec fichier image Cloudinary	HTTP 201, image renseignée	✓
PO-03	Post vide	message: "	HTTP 400 validation Zod	✓
PO-04	Liker un post	PUT /api/private/reaction, type LIKE	HTTP 200, reaction créée	✓
PO-05	Changer de réaction	PUT /api/private/reaction, type LOVE	HTTP 200, réaction mise a jour (upsert)	✓
PO-06	Supprimer sa réaction	DELETE /api/private/reaction/[id]	HTTP 200, réaction supprimée	✓
PO-07	Commenter un post	POST /api/private/post/[id]/comments	HTTP 201, commentaire créé,	✓

7.6 GROUPES ET EVENEMENTS

ID	CAS DE TEST	ENTRÉE SIMULEE	RESULTAT ATTENDU	✓
GR-01	Créer un groupe	POST /api/private/groups/create	HTTP 200, créateur = owner	✓
GR-02	Rejoindre un groupe public	POST /api/private/groups/[id]/members	HTTP 200	✓
GR-03	Rejoindre groupe prive sans invitation	POST sans invitation	HTTP 403 Forbidden	✓
GR-04	Créer un évènement de groupe	POST /api/private/groups/[id]/events, groupId valide	HTTP 200, event lié au groupe	✓
GR-05	RSVP a un évènement	POST /api/private/events/[id]/rsvp, status YES	HTTP 200, participant ajouté	✓
GR-06	Changer son RSVP	POST /api/private/events/[id]/rsvp, status MAYBE	HTTP 200, RSVP mis à jour	✓

7.7 RESULTAT GLOBAL – SYNTHÈSE DES TESTS

A l'issue de la campagne de tests, voici le bilan global de la couverture :

DOMAINE	CAS DE TEST	PASSES	ECHECS	PARTIELS
Authentification (AU)	11	11	0	0
Posts et interactions (PO)	7	7	0	0
Messagerie chat (CH)	8	8	0	0
Groupes et événements (GR)	6	6	0	0
Sécurité (SE)	6	6	0	0
TOTAL	38	38	0	0

```

D4rk Lik3 .../social-network  main 13:39
PASS ui __tests__/ui/button.test.tsx
  Button component
    ✓ renders children text (41 ms)
    ✓ is disabled when disabled prop is set (5 ms)
    ✓ calls onClick handler when clicked (12 ms)
    ✓ does not call onClick when disabled (7 ms)
    ✓ applies variant class correctly (7 ms)

Test Suites: 1 passed, 1 total
Tests:       5 passed, 5 total
Snapshots:   0 total
Time:        0.473 s, estimated 1 s
Ran all test suites matching /button/i.

```

Exécution Jest — 5 tests unitaires automatisés passent au vert (composant Button, React Testing Library). 2 tests d'intégration backend sur les routes register et login avec vraie base PostgreSQL. Extension prévue : 10 routes critiques + 5 parcours E2E Playwright.

8. SECURITE ET RGPD

8.1 MESURES DE SECURITE IMPLEMENTEES

MENACE / RISQUE	MESURE MISE EN PLACE
Identifiants en clair	bcrypt avec salt rounds=12, jamais de mot de passe en clair
Vol de session (XSS)	Cookie httpOnly, Secure, SameSite=Lax
CSRF	SameSite=Lax sur les cookies
Injection SQL	Prisma ORM avec requêtes paramétrées systématiques
Accès non autorisé	Middleware JWT sur toutes les routes protégées, vérification ownership
Exposition des données	Sélection Prisma explicite (jamais SELECT *), DTOs typés
Upload malveillant	Validation MIME type Cloudinary, taille max 50MB
Secrets exposés	Variables .env non versionnées, validation au démarrage
Énumération utilisateurs	Même message d'erreur pour email/pwd invalides
Token JWT longue durée	JWT exp : 480min (8h), cookie maxAge, suppression du cookie à la déconnexion

8.2 CONFORMITE RGPD

Même si le projet est à vocation pédagogique, nous avons appliqué les principes du RGPD dès la conception (privacy by design) :

PRINCIPE RGPD	IMPLEMENTATION DANS LE PROJET
Minimisation des données	Seules les données nécessaires sont collectées (pas de données de géolocalisation)
Droit d'accès	Profil utilisateur accessible et modifiable depuis l'interface
Sécurité des données	Chiffrement en transit (HTTPS/TLS), hashage des mots de passe, pas de données sensibles en logs

Headers de sécurité HTTP

```
next.config.ts X
next.config.ts > securityHeaders
You, yesterday | 4 authors (You and others)
1 import type { NextConfig } from "next";
2
3 const securityHeaders = [
4   { key: "X-Frame-Options", value: "DENY" },
5   { key: "X-Content-Type-Options", value: "nosniff" },
6   { key: "Referrer-Policy", value: "strict-origin-when-cross-origin" },
7   { key: "Permissions-Policy", value: "camera=(), microphone=(), geolocation=()" },
8   {
9     key: "Strict-Transport-Security",
10    value: "max-age=63072000; includeSubDomains; preload",
11  },
12  {
13    key: "Content-Security-Policy",
14    value: [
15      "default-src 'self'",
16      "script-src 'self' 'unsafe-inline' 'unsafe-eval'",
17      "style-src 'self' 'unsafe-inline'",
18      "img-src 'self' data: blob: https://res.cloudinary.com https://images.pexels.com
19      "font-src 'self'",
20      "connect-src 'self' https://\*.upstash.io https://api.cloudinary.com",
21      "media-src 'self' blob:",
22      "frame-ancestors 'none'",
23    ].join("; "),
24  },
25 ];
26
```

Validation des entrées — Schéma Zod

```

TS create.ts x
src > lib > schemas > post > TS create.ts > CreatePostSchema
pcaboor, 9 months ago | 3 authors (clecart and others)
1 import { z } from "zod";
2 import { Visibility } from "@prisma/client";
3
4 export const MAX_FILE_SIZE = 10 * 1024 * 1024;
5
6 export const ACCEPTED_FILE_TYPES = [
7   "image/jpeg",
8   "image/png",
9   "image/webp",
10  "image/gif",
11  "image/avif",
12  "video/mp4",
13  "video/webm",
14  "video/ogg",
15  "video/quicktime",
16 ];
17
18 export const CreatePostSchema = z
19   .object({
20     content: z.string().min(1, "Content is required"),
21     media: z
22       .instanceof(File)
23       .optional()
24       .refine((file) => !file || file.size <= MAX_FILE_SIZE, {
25         message: "File must be less than 10MB.",
26       })
27       .refine((file) => !file || ACCEPTED_FILE_TYPES.includes(file.type), {
28         message:
29           "Unsupported file format. Allowed: GIF, AVIF, JPEG, PNG, WEBP, MP4, WEBM, OGG, MOV.",
30       }),
31     visibility: z.nativeEnum(Visibility).default(Visibility.PUBLIC),
32   })
33   .describe("CreatePostSchema");
34
35 export type CreatePost = z.infer<typeof CreatePostSchema>;

```

```

TS base.ts x
src > lib > schemas > reaction > TS base.ts > ...
Gabriel K, 8 months ago | 3 authors (Gabriel K and others)
1 import { z } from "zod";
2 import { ReactionUserSchema } from "../stories/user";
3
4 export const ReactionTypeSchema = z.enum([
5   "LIKE",
6   "DISLIKE",
7   "LOVE",
8   "LAUGH",
9   "SAD",
10  "ANGRY",
11  "WOW"
12 ]).describe('ReactionTypeSchema');
13
14 export const ReactionSchema = z
15   .object({
16     id: z.string(),
17     type: ReactionTypeSchema,
18     user: ReactionUserSchema,
19   })
20   .describe("ReactionSchema");
21
22 export type ReactionType = z.infer<typeof ReactionTypeSchema>;
23 export type Reaction = z.infer<typeof ReactionSchema>;
24

```

9. DEPLOIEMENT

9.1 INFRASTRUCTURE DOCKER – DEVELOPPEMENT LOCAL

Pour garantir la reproductibilité de l'environnement de développement, nous avons mis en place un fichier docker-compose.yml qui orchestre le service applicatif. La base de données (Neon) et Redis (Upstash) sont des services externes, aucun service postgres ou redis n'est lancé en local.

```

docker-compose.yml x
docker-compose.yml > {} services > {} app > {} ports
docker-compose.yml - The Compose specification establishes a standard for the definition of multi-container platform-agnostic applications (compose-spec.json)
1 services:
  > Run Service
2 app:
3   profiles: ["prod"] # ☒ ne démarre qu'avec --profile prod
4   build:
5     context: .
6     dockerfile: Dockerfile
7   restart: unless-stopped
8   env_file:
9     - .env
10  environment:
11    DATABASE_URL: ${DATABASE_URL}
12    NODE_ENV: production
13    NEXT_TELEMETRY_DISABLED: "1"
14  ports:
15    - "3000:3000"
16  Gabriel K, 8 months ago via PR #118 · chore(compose): switch to Neon DATABASE_URL and...
  > Run Service
17 app-dev:
18   profiles: ["dev"] # ☒ ne démarre qu'avec --profile dev
19   image: oven/bun:1
20   working_dir: /app
21   restart: unless-stopped
22   env_file:
23     - .env
24   environment:
25    DATABASE_URL: ${DATABASE_URL}
26    NODE_ENV: development
27    NEXT_TELEMETRY_DISABLED: "1"
28    WATCHPACK_POLLING: "true"
29    CHOKIDAR_USEPOLLING: "true"
30   command: >
31     sh -c "
32       bun install &&
33       bunx prisma generate &&
34       bunx prisma migrate deploy &&
35       bun run dev
36     "
37   ports:
38     - "3000:3000"
39   volumes:
40     - ./app
41     - app-node-modules:/app/node_modules
42     - app-next-cache:/app/.next
43   stdin_open: true
44   tty: true
45
46 volumes:
47   app-node-modules:
48   app-next-cache:

```


9.2 CI/CD ET MISE EN PRODUCTION

Le déploiement repose sur une image Docker multi-stage (Bun + Next.js). Le script **build.sh** orchestre le build et le lancement du conteneur localement. En production, le déploiement est déclenché automatiquement via l'intégration Git native de Versel sur push vers *main*. L'analyse de qualité de code est assurée par SonarQube (*sonar-project.properties*).

```
Dockerfile X
Dockerfile > ...
Gabriel K, 8 months ago | 2 authors (arocchet and one other)
1 # Étape 1 : build de l'app
2 FROM oven/bun:1 AS builder
3
4 WORKDIR /app
5
6 # Ensure OpenSSL is available for Prisma (detects and links engines correctly)
7 RUN apt-get update -y \
8     && apt-get install -y --no-install-recommends openssl ca-certificates \
9     && rm -rf /var/lib/apt/lists/*
10
11 # Copie uniquement les fichiers nécessaires à l'installation
12 COPY package.json bun.lock ./
13
14 # Installe TOUTES les dépendances (y compris dev)
15 RUN bun install
16
17 # Copie le reste du code source
18 COPY . .
19
20 # Prisma generate
21 RUN bunx prisma generate
22
23 # Build Next.js
24 RUN bun run build
25
26 # Étape 2 : image finale allégée
27 FROM oven/bun:1 AS runner
28
29 WORKDIR /app
30
31 # OpenSSL also required at runtime for Prisma engines
32 RUN apt-get update -y \
33     && apt-get install -y --no-install-recommends openssl ca-certificates \
34     && rm -rf /var/lib/apt/lists/*
35
36 # Copie uniquement le build et les node_modules depuis le builder
37 COPY --from=builder /app/node_modules ./node_modules
38 COPY --from=builder /app/.next ./next
39 COPY --from=builder /app/package.json ./package.json
40 COPY --from=builder /app/public ./public
41 COPY --from=builder /app/prisma ./prisma
42
43 ENV NODE_ENV=production
44 ENV NEXT_TELEMETRY_DISABLED=1
45 EXPOSE 3000
46
47 # Apply pending Prisma migrations on startup, then launch Next.js
48 CMD ["sh", "-c", "bunx prisma migrate deploy && bun run start"]
```

Variables d'environnement de production

VARIABLE	RÔLE	SOURCE
DATABASE_URL	Connexion PostgreSQL Neon	Neon Dashboard
UPSTASH_REDIS_REST_URL	Redis temps réel Upstash	Upstash Console
UPSTASH_REDIS_REST_TOKEN	Token auth Redis	Upstash Console
JWT_SECRET	Clé de signature JWT (256 bits)	Générée via openssl rand
3 variables séparées: <i>CLOUDINARY_CLOUD_NAME,</i> <i>CLOUDINARY_API_KEY,</i> <i>CLOUDINARY_API_SECRET</i>	URL + clé API Cloudinary	Cloudinary Dashboard
CLIENT_ID	OAuth Google Client ID	Google Cloud Console
<i>CLIENT_SECRET</i>	OAuth Google Secret	Google Cloud Console
OAUTH_TOKEN_ENCRYPTION_KEY	Clé de chiffrement token OAuth	Générée via openssl rand

10. RECHERCHE TECHNIQUE PERSONNELLE

10.1 APPROCHE DE VEILLE

Le projet m'a conduit à explorer des domaines techniques souvent peu abordés en formation classique. Ma démarche de veille est structurée autour de quatre axes : lire la documentation officielle, tester en pratique, comparer les approches alternatives, et m'appuyer sur des retours d'expérience de la communauté.

THEMATIQUE	PROBLEME RENCONTRE	SOLUTION EXPLOREE
SSE vs WebSockets	Vercel ne supporte pas les WebSockets durables en serverless	SSE + Redis polling (GET toutes les 500ms)
React Server Components	Comprendre la frontière client/serveur dans App Router	Lecture RFC RSC, expérimentation avec Suspense et streaming SSR
Pagination offset	Pagination par offset sur grandes tables	skip/take Prisma avec filtre de visibilité intégré
Prisma N+1	Requêtes multiples pour les relations imbriquées	include vs select Prisma, dataloader pattern, requêtes agrégées
JWT vs Sessions DB	Choisir entre stateless JWT et sessions stockées	JWT httpOnly pour simplicité serverless, refresh via reconnexion 7j
Cloudinary upload	Upload sécurisé sans exposer les clés API côté client	Signed upload preset : le serveur génère une signature temporaire

10.2 ANALYSE COMPARATIVE – TECHNOLOGIES TEMPS REEL

Le choix de la solution temps réel est l'une des décisions architecturales les plus importantes du projet. Nous avons comparé trois approches.

CRITERE	WebSockets	SSE + Redis	Long Polling
Compatibilité Vercel/Edge	✗ Non	✓ Oui	✓ Oui
Bidirectionnel natif	✓ Oui	✗ Non (push only)	✗ Non
Latence	< 10ms	< 100ms	1-3s
Complexité implémentation	Élevée (serveur dédié)	Moyenne	Faible
Scalabilité serverless	Difficile	✓ Excellente	Correcte
Choix retenu	—	✓ CHOISI	Fallback

10.3 SOURCES MOBILISEES

- Documentation officielle : Next.js docs, Prisma docs, Upstash Redis, Vercel documentation
- Articles techniques : Lee Robinson (Vercel), Josh W. Comeau, Kent C. Dodds (testing)
- Conférences et talks : React Conf 2024 (RSC deep dive), Next.js Conf
- GitHub : Issues Prisma, discussions Next.js App Router RFC, exemples officiels
- Communauté : Discord Next.js, r/nextjs, Stack Overflow pour les cas limites
- Expérimentation directe : Branches de test, prototypes isolés avant intégration

11. BILAN RNCP ET CONCLUSION

11.1 COUVERTURE DES COMPETENCES RNCP 37873

Le projet Konekt couvre l'intégralité des compétences attendues du titre professionnel CDA (RNCP 37873). Voici la correspondance détaillée.

COMPETENCE RNCP	REALISATION CONCRETE	NIVEAU
Développer des composants d'accès aux données	18 modèles Prisma, 60+ requêtes optimisées, migrations, seed	✓ Validé
Concevoir une IHM adaptée au contexte	110+ composants React, responsive mobile-first, design system Tailwind	✓ Validé
Développer la partie back-end d'une interface web	60+ endpoints API REST, Next.js API Routes, validation Zod, gestion erreurs	✓ Validé
Mettre en place une base de données	PostgreSQL via Prisma, 18 entités, 40+ relations, contraintes, index	✓ Validé
Développer des composants dans le langage de l'environnement	TypeScript strict, composants Server et Client, hooks React, utilitaires	✓ Validé
Préparer et exécuter les plans de tests	7 tests automatisés Jest, 38 jeux d'essai documentés	✓ Validé
Préparer et documenter le déploiement	Docker multi-stage, déploiement Vercel, SonarQube	✓ Validé
Contribuer à la mise en production	Déploiement Vercel, migration Prisma prod, variables d'environnement sécurisées	✓ Validé
Concevoir une application en couches	App Router Next.js, séparation UI/API/Data, middleware, services	✓ Validé
Utiliser des composants d'accès aux données	Prisma client type-safe, Redis Upstash SDK, Cloudinary SDK	✓ Validé
Développer une partie front-end sécurisée	JWT httpOnly, SameSite=Lax validation côté client et serveur, HTTPS	✓ Validé

11.2 BILAN PERSONNEL

Ce projet représente pour moi bien plus qu'un exercice de certification. C'est la preuve que ma reconversion est aboutie et que je suis capable de livrer une application complexe, sécurisée et documentée, de A à Z.

La complexité du projet — 18 modèles de données, des flux temps réel, une API de 60+ endpoints, des contraintes d'hébergement serverless — m'a forcé à aller chercher des solutions que la formation seule n'enseigne pas. J'ai appris à résoudre des problèmes réels : optimiser des requêtes qui créaient des timeouts, implémenter du temps réel sans WebSocket, concevoir un système de notification multi-types sans accumuler la dette technique.

Ma reconversion depuis l'industrie m'a apporté une perspective précieuse : dans les métiers techniques, la fiabilité compte plus que la sophistication. Un code maintenable, testé, documenté et déployable vaut infiniment plus qu'une *feature* brillante fragile. C'est avec cette conviction que j'aborde ma recherche d'alternance Bac+5.

11.3 FORCES ET AXES D'AMELIORATION

 POINTS FORTS	 AXES D'AMELIORATION
Architecture API robuste et scalable	Tests unitaires encore insuffisants
Maîtrise de Prisma et PostgreSQL	Rate limiting à étendre (chat/send)
Système temps réel SSE + Redis fonctionnel	WebSockets à explorer en environnement dédié
Couverture de test documentée	Internationalisation (i18n) absente
TypeScript strict bout en bout	Tests de charge non réalisés
Déploiement Docker + Vercel opérationnel	Accessibilité (WCAG) partiellement couverte

11.4 VISION ET PERSPECTIVES

Ce projet marque la fin d'une première étape et le début d'une trajectoire que j'ai décidée de poursuivre en alternance Bac+5 Chef de projet, Architecture et Green IT. Je souhaite approfondir mes compétences en architecture de systèmes distribués, en optimisation de performances à l'échelle et en pratiques DevOps avancées, avec une attention particulière aux enjeux de sobriété numérique.

Les domaines qui m'attirent particulièrement pour la suite sont les architectures event-driven (Kafka, RabbitMQ), l'observabilité (OpenTelemetry, Grafana), la sécurité applicative approfondie ainsi que les approches Green IT visant à concevoir des systèmes plus durables et efficaces.

Ma reconversion depuis l'industrie m'a appris que la fiabilité n'est pas un luxe — c'est le standard minimum.

"Ce dossier est le reflet d'une conviction : on n'apprend pas à développer en lisant, on apprend en construisant quelque chose qui résiste à la réalité."
